PRAXISHANDBUCH

Digitale Forensik

Digitale Spuren sichern, analysieren und Beweise nachvollziehbar dokumentieren

```
$ sha256sum beweis.raw
3f9a1c...e07b  beweis.raw  (SHA-256)
$ sha256sum -c hashes.txt
beweis.raw: OK
```

SPUREN SICHERN. ZUSAMMENHÄNGE ERKENNEN. FAKTEN DOKUMENTIEREN.

Mit Übungen, Diagrammen, Glossar und Index · Autor: Eugen Becker

Inhaltsverzeichnis

Vorwort 1

Warum digitale Forensik an Bedeutung gewinnt 1
Digitale Spuren im Alltag 1
Bedeutung für Unternehmen und Behörden 1
Zusammenhang zwischen IT-Sicherheit und Forensik 2
Forensik im Blue Team 2
Für wen ist dieses Handbuch? 2
Aufbau des Buches 2
Hinweis zu den Befehlen 3
Voraussetzungen für die Praxisübungen 3

Kapitel 1: Einführung in die digitale Forensik 4

Was ist digitale Forensik? 4
Teilgebiete 5
Ziele einer Untersuchung 5
Digitale Beweismittel 5
Flüchtige und nichtflüchtige Daten 6
Digitale Spuren 6
Abgrenzung zu Incident Response und Administration 7
Typische Einsatzgebiete 7
Grenzen der digitalen Forensik 7
Kontrollfragen 8
Lernzielkontrolle 9

Kapitel 2: Der forensische Untersuchungsprozess 10

Ein Ausgangsbeispiel 10
Die Phasen im Überblick 10
Vorbereitung und Untersuchungsauftrag 11
Identifikation der Beweismittel 12
Sicherung: Acquisition und Preservation 12
Untersuchung und Analyse 12
Dokumentation und Bericht 13
Beobachtung und Schlussfolgerung trennen 13

Umgang mit Fehlern 13
Typische Fehler 13
Übung A: Untersuchungsauftrag formulieren 14
Übung B: Beobachtung oder Schlussfolgerung? 15
Kontrollfragen 15
Lernzielkontrolle 16

Kapitel 3: Beweissicherung und Chain of Custody 17
Was ist ein Beweismittel, und was macht es belastbar? 17
Identifikation 17
Zustand dokumentieren 18
Zeitangaben 18
Beweismittellabel und Nummerierung 18
Verantwortliche Personen 18
Chain of Custody 19
Manipulationsschutz und Lagerung 19
Schreibschutz 19
Physische und logische Sicherung 20
Umgang mit beschädigten Datenträgern 20
Typische Fehlerquellen 20
Praxis: Beweismittel erfassen 21
Kontrollfragen 23
Lernzielkontrolle 24

Kapitel 4: Forensische Datenträgerimages 25
Was ist ein forensisches Image? 25
Backup und Image 25
Bit-für-Bit-Kopie 26
Physische und logische Sicherung 26
Datenträgerstruktur: MBR und GPT 26
Unbelegter Speicher und Slack Space 27
Schreibschutz 27
Imageformate 27
Metadaten und Protokoll 28
Integritätsprüfung 28
Typische Fehler 28

Praxis: Testdatenträger, Image und Prüfung 29
Kontrollfragen 32
Lernzielkontrolle 32

Kapitel 5: Hashwerte und Integritätsprüfung 34
Was ein Hashwert ist 34
Verfahren und ihr Status 34
Was ein Hashnachweis belegt und was nicht 35
Hashwerte vor und nach der Sicherung 36
Hashlisten für Verzeichnisse 36
Zugriffszeiten und Hashberechnung 36
Typische Fehler 37
Praxis: Hashwerte berechnen und prüfen 37
Kontrollfragen 41
Lernzielkontrolle 41

Kapitel 6: Windows-Forensik 43
Die Spuren eines Windows-Systems 43
Ereignisprotokolle 44
Registry 45
Prefetch 46
USB-Gerätehistorie 47
Weitere Artefakte, nur mit Grenzen 48
Praxis: Windows-Spuren auswerten 48
Typische Fehler 51
Kontrollfragen 52
Lernzielkontrolle 52

Kapitel 7: Linux-Forensik 53
Einordnung 53
Systemprotokolle und das Journal 54
Anmeldungen und Authentifizierung 55
Befehlshistorie 55
Benutzer, Gruppen und Anmeldedaten 56
Geplante Aufgaben 56
Dienste und Autostart 56

Paketinstallationen 57
Dateimetadaten 57
Zeitgleiche Ereignisse im Journal 58
Typische Fehler 58
Praxis: Linux-Spuren auswerten 58
Kontrollfragen 62
Lernzielkontrolle 63

Kapitel 8: Timeline-Analyse 64
Was eine Timeline ist 64
Die vier Zeitstempel einer Datei 65
Zeitzonen, UTC und lokale Zeit 66
Uhrabweichungen und NTP 66
Ereigniskorrelation 67
Grenzen der Timeline-Analyse 67
Praxis: Eine Timeline erstellen 67
Typische Fehler 71
Kontrollfragen 72
Lernzielkontrolle 73

Kapitel 9: Digitale Netzwerkforensik 74
Was Netzwerkspuren sind 74
IP-Adressen, Ports und Protokolle 75
DNS: Namensauflösung als Spur 75
HTTP und TLS 76
Verbindungen und Dauer 76
Grenzen der Netzwerkforensik 76
Typische Fehler 76
Praxis: Netzwerkspuren in einem eigenen Testmitschnitt 77
Kontrollfragen 81
Lernzielkontrolle 82

Kapitel 10: Praxisprojekt – Eine simulierte Untersuchung 83
Der Fall 83
Umgebung 84
Aufgabe 1: Untersuchungsauftrag formulieren 84

Aufgabe 2: Beweismittel erfassen 85
Aufgabe 3: Image erstellen und prüfen 85
Aufgabe 4: Partitionsstruktur lesen 86
Aufgabe 5: Systemspuren auswerten 86
Aufgabe 6: Timeline erstellen 87
Aufgabe 7: Netzwerkmitschnitt auswerten 88
Abschlussbericht 88
Qualitätsprüfung des Projekts 89
Kontrollfragen 89
Lernzielkontrolle 90

Anhang 91

Forensischer Spickzettel 91
Linux-Befehle dieses Buches 91
Windows-PowerShell-Befehle dieses Buches 93
Hash-Befehle im Überblick 93
Wireshark- und tshark-Filter 94
Formular: Chain of Custody 94
Formular: Übergabeprotokoll 95
Beweismittelverzeichnis 95
Untersuchungsprotokoll 95
Muster eines Abschlussberichts 95
Glossar 96
Quellenverzeichnis 98
Verzeichnis der Befehlsbeispiele 99
Index 100

Vorwort

Wenn ein Rechner ungewöhnlich langsam wird, eine Datei verschwunden ist oder eine Anmeldung in der Nacht auftaucht, in der niemand am Gerät gesessen hat, stellt sich schnell dieselbe Frage: Was ist hier eigentlich passiert? Die digitale Forensik beantwortet sie, indem sie digitale Spuren sichert, auswertet und so festhält, dass andere die Ergebnisse nachvollziehen können.

Warum digitale Forensik an Bedeutung gewinnt

Immer mehr Abläufe finden auf Rechnern statt: Buchhaltung, Kommunikation, Steuerung von Maschinen, Zugang zu Gebäuden. Damit entstehen auch immer mehr Spuren. Gleichzeitig nehmen Sicherheitsvorfälle zu, bei denen Angreifer Konten übernehmen, Daten abfließen lassen oder Schadsoftware dauerhaft installieren. Wer dann nicht belegen kann, was geschehen ist, weiß oft auch nicht, was er beheben muss.

Digitale Spuren im Alltag

Auch privat hinterlässt fast jede Handlung Spuren. Ein Smartphone protokolliert Standorte, ein Browser speichert Verläufe, ein Betriebssystem schreibt Zeitstempel in Dateien und Protokolle. Die meisten dieser Spuren entstehen als Nebenprodukt und nicht absichtlich. Genau darin liegt der Wert der Forensik, aber auch ihre Schwierigkeit, denn Nebenprodukte werden selten mit einer Untersuchung im Hinterkopf erzeugt.

Bedeutung für Unternehmen und Behörden

Unternehmen brauchen forensische Fähigkeiten, um Vorfälle richtig einzuordnen, Schäden zu begrenzen und gegenüber Kunden, Versicherungen oder Aufsichtsbehörden begründete Aussagen zu treffen. Behörden untersuchen Datenträger in Ermittlungsverfahren. In beiden Fällen gilt: Ein Ergebnis ist nur so viel wert wie der Weg, auf dem es entstanden ist.

Zusammenhang zwischen IT-Sicherheit und Forensik

IT-Sicherheit soll Vorfälle verhindern, Forensik soll sie aufklären, wenn die Prävention versagt hat. Beide Bereiche greifen ineinander. Protokolle, die die Sicherheitsabteilung für die Überwachung braucht, sind meist auch die wichtigsten Quellen einer späteren Untersuchung. Wer also schon bei der Einrichtung auf ausreichende Protokollierung achtet, erleichtert spätere Untersuchungen erheblich.

Forensik im Blue Team

Blue-Team-Analysten verteidigen Netzwerke und erkennen Angriffe im laufenden Betrieb. Sie sehen oft zuerst eine Auffälligkeit, etwa einen ungewöhnlichen Prozess oder einen verdächtigen Netzwerkverkehr. Die forensische Methode hilft ihnen, aus einem Verdacht eine belegbare Feststellung zu machen, bevor Maßnahmen ergriffen werden, die Spuren verändern.

Für wen ist dieses Handbuch?

Das Buch richtet sich an Leser ohne forensische Vorkenntnisse, die verstehen wollen, wie eine saubere Untersuchung abläuft. Angehende Fachinformatiker, Systemadministratoren und Einsteiger in die IT-Sicherheit finden hier das methodische Grundgerüst. Wer schon forensisch arbeitet, wird vor allem die Kapitel zu Beweissicherung, Hashwerten und Timelines als Auffrischung nutzen können.

Aufbau des Buches

Dieses Handbuch konzentriert sich auf den methodischen Kern: Einführung und Untersuchungsprozess (Kapitel 1 bis 3), die Sicherung von Datenträgern (Kapitel 4 und 5), die Auswertung von Windows- und Linux-Systemen (Kapitel 6 und 7), die Timeline (Kapitel 8), Netzwerkspuren (Kapitel 9) sowie ein zusammenhängendes Praxisprojekt (Kapitel 10). Memory Forensics, Mobile- und Cloud-Forensik, die Vorstellung einzelner Werkzeuge und ausführliche Fallstudien sind bewusst nicht enthalten.

Hinweis zu den Befehlen

Die Befehle in diesem Buch sind nach der Dokumentation der jeweiligen Werkzeuge zusammengestellt. Sie wurden vor der Veröffentlichung nicht praktisch getestet. Prüfe Befehle und Ausgaben deshalb vor dem Einsatz an deinem eigenen Testsystem, und vergleiche Ergebnisse immer mit der offiziellen Dokumentation der verwendeten Version. Bei forensischer Arbeit gilt das ohne Ausnahme: Ein Befehl, dessen Wirkung du nicht kennst, gehört nicht auf ein Beweismittel.

Voraussetzungen für die Praxisübungen

Du brauchst eine Linux-VM mit einer aktuellen Distribution und eine Windows-11-VM als Testsystem. Alle Übungen finden ausschließlich auf eigenen oder ausdrücklich freigegebenen Systemen und mit selbst erzeugten Testdaten statt. Die Werkzeuge installierst du über die Paketverwaltung beziehungsweise die offiziellen Downloadseiten der Hersteller.

Kapitel 1: Einführung in die digitale Forensik

Lernziele

- Du beschreibst, was digitale Forensik untersucht und welche Teilgebiete es gibt.
- Du unterscheidest flüchtige von nichtflüchtigen Daten und begründest, warum die Reihenfolge der Sicherung wichtig ist.
- Du grenzt forensische Untersuchung gegen Incident Response und Systemadministration ab.

Gut zu wissen

Dieses Kapitel enthält bewusst keine Befehle. Die ersten Werkzeuge folgen in Kapitel 4.

Was ist digitale Forensik?

Digitale Forensik wendet methodische Verfahren an, um digitale Daten so zu sichern, zu untersuchen und zu dokumentieren, dass die Ergebnisse überprüfbar sind. Das Wort stammt vom lateinischen *forensis* ab, das zum öffentlichen Forum gehört und an die Verhandlung vor Gericht erinnert. Der Begriff meint also kein Hacker-Handwerk, sondern ein Verfahren, dessen Ergebnisse auch Dritte nachvollziehen können müssen.

Teilgebiete

Die digitale Forensik gliedert sich nach dem Untersuchungsgegenstand. **Computer Forensics** befasst sich mit Datenträgern und Betriebssystemen von Arbeitsplatzrechnern und Servern. **Mobile Forensics** untersucht Smartphones und Tablets. **Network Forensics** wertet Datenverkehr und Protokolle von Netzwerkkomponenten aus. **Memory Forensics** analysiert den Arbeitsspeicher, und **Cloud Forensics** betrifft Dienste, deren Daten nicht auf dem eigenen Gerät liegen. Die Grenzen zwischen den Bereichen sind fließend: Ein Vorfall auf einem Server berührt oft mehrere davon gleichzeitig.

Ziele einer Untersuchung

Eine Untersuchung beantwortet meist eine konkrete Frage. Wann hat sich ein Konto angemeldet? Welche Datei wurde verändert oder abgeflossen? Auf welchem Weg gelangte ein Schadprogramm auf das System? Die Antworten müssen auf Tatsachen beruhen, die sich aus den gesicherten Daten ablesen lassen. Welche rechtlichen Folgen sich daraus ergeben, entscheiden Gerichte, Behörden oder Auftraggeber. Die Unterscheidung zwischen technischem Befund und rechtlicher Bewertung zieht sich durch das ganze Buch und wird in Kapitel 3 und 14 vertieft.

Digitale Beweismittel

Ein digitales Beweismittel ist ein Datenbestand, der mit einem Vorfall in Verbindung stehen kann. Dazu gehören Festplatten, USB-Sticks, Smartphones, Protokolldateien, Netzwerkmitschnitte und Speicherabbilder. Ob ein Datenbestand als Beweismittel überhaupt belastbar ist, hängt davon ab, wie er gesichert und dokumentiert wurde. Ein Datenträger mit unklarer Herkunft bleibt schwach, auch wenn seine Inhalte eindeutig wirken.

Flüchtige und nichtflüchtige Daten

Manche Daten verschwinden beim Abschalten, andere bleiben erhalten. Den Inhalt des Arbeitsspeichers, offene Netzwerkverbindungen oder laufende Prozesse nennt man flüchtig. Dateien auf einer Festplatte oder SSD sind nichtflüchtig, können sich aber ebenfalls ändern, solange das System läuft. Die RFC 3227 der Internet Engineering Task Force, eine Empfehlung zur Sammlung und Archivierung von Beweismaterial, legt deshalb eine Reihenfolge fest, in der Daten gesichert werden sollten: zuerst die flüchtigsten.

Diagramm 1.1: Sicherungsreihenfolge (vereinfacht nach RFC 3227)

```
1. Register und Cache
2. Arbeitsspeicher, Prozesstabelle, Netzwerkverbindungen, ARP-Tabelle
3. temporäre Dateisysteme
4. Datenträger (Festplatte, SSD)
5. entfernte Protokollierung
6. Archivmedien
```

So wird vermieden, dass ein Neustart oder ein Stromausfall flüchtige Daten zerstört, bevor sie gesichert sind.

Digitale Spuren

Spuren entstehen meist als Nebenprodukt. Ein Betriebssystem schreibt Zeitstempel, Ereignisprotokolle und Verlaufsdateien, ohne dass ein Benutzer das bewusst auslöst. Darin liegt der Wert der Forensik, aber auch ihre Schwäche: Viele Spuren lassen sich von Angreifern oder fehlerhafter Software verändern, und Speicherbereiche werden im laufenden Betrieb wiederverwendet. Kapitel 7 und 8 zeigen, welche Spuren unter Windows und Linux typischerweise anfallen.

Abgrenzung zu Incident Response und Administration

Incident Response ist der übergeordnete Prozess, mit dem eine Organisation auf einen Sicherheitsvorfall reagiert: erkennen, eindämmen, beseitigen, wiederherstellen. Digitale Forensik ist ein Werkzeug innerhalb dieses Prozesses und klärt vor allem Ursache und Ausmaß. Die Systemadministration verfolgt ein anderes Ziel, nämlich den Betrieb aufrechtzuerhalten. Dabei werden Systeme verändert, Protokolle rotiert und Dateien gelöscht. Genau solche Eingriffe können Spuren vernichten, die eine Untersuchung braucht. Bei einem vermuteten Vorfall sollte die Administration deshalb die Untersuchung früh einbeziehen und Eingriffe mit ihr abstimmen.

Typische Einsatzgebiete

Unternehmen setzen Forensik ein, wenn ein Konto übernommen wurde, Daten abgeflossen sind oder Schadsoftware auftaucht. Behörden untersuchen Datenträger in Ermittlungsverfahren. Auch in arbeitsrechtlichen oder vertraglichen Auseinandersetzungen spielen digitale Spuren eine Rolle. Die meisten Untersuchungen beginnen allerdings mit einem Verdacht in der IT-Abteilung und nicht mit einem Gerichtsverfahren.

Grenzen der digitalen Forensik

Forensik kann nur finden, was Spuren hinterlassen hat. Vollständig gelöschte Daten, überschriebene Protokolle und verschlüsselte Container ohne Schlüssel setzen harte Grenzen. Eine Zeitangabe in einem Protokoll ist außerdem nur so verlässlich wie die Uhr, die sie geschrieben hat; Kapitel 8 behandelt dieses Problem ausführlich. Ein technischer Befund ist zudem noch keine Aussage über Schuld: Dass eine Aktion von einem bestimmten Konto ausging, bedeutet nicht, dass die Person am Gerät die Aktion selbst ausgelöst hat.

Zusammenfassung

- Digitale Forensik sichert, untersucht und dokumentiert digitale Daten so, dass Ergebnisse überprüfbar sind.
- Flüchtige Daten gehen beim Abschalten verloren und müssen deshalb zuerst gesichert werden.
- Forensik klärt Ursache und Ausmaß, Incident Response steuert die Reaktion, Administration hält den Betrieb am Laufen.
- Technische Befunde und rechtliche Bewertungen sind getrennt zu betrachten.

Kontrollfragen

1. Was ist der Unterschied zwischen flüchtigen und nichtflüchtigen Daten? (leicht)
2. Nenne zwei Teilgebiete der digitalen Forensik. (leicht)
3. Warum sollten flüchtige Daten vor einem Neustart gesichert werden? (mittel)
4. Worin unterscheiden sich Incident Response und digitale Forensik? (mittel)
5. Ein Administrator hat auf einem betroffenen Server Logdateien gelöscht, um Speicherplatz zu gewinnen. Welche Folgen hat das für die Untersuchung? (schwer)

Lösungshinweise: 1. Flüchtige Daten verschwinden beim Abschalten, nichtflüchtige bleiben auf dem Datenträger erhalten. 2. Zum Beispiel Computer Forensics, Mobile Forensics, Network Forensics, Memory Forensics oder Cloud Forensics. 3. Weil ein Neustart den Arbeitsspeicher und andere flüchtige Zustände unwiederbringlich verändert oder löscht. 4. Incident Response steuert den gesamten Reaktionsprozess, digitale Forensik liefert die belegbaren Erkenntnisse über Ursache und Ausmaß innerhalb davon. 5. Gelöschte Protokolle können verlorene Zeiträume und Ereignisse bedeuten; zusätzlich werden Zeitabläufe unvollständig, und der Eingriff muss selbst dokumentiert werden, damit die Lücke erklärbar bleibt.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] die Kontrollfragen 1 bis 4 sicher beantworten kannst,
- [ ] die Sicherungsreihenfolge aus Diagramm 1.1 in eigenen Worten wiedergeben kannst,
- [ ] erklären kannst, warum Administration und Untersuchung abgestimmt werden müssen.

Kapitel 2: Der forensische Untersuchungsprozess

Lernziele

- Du beschreibst die Phasen einer forensischen Untersuchung und ihre Reihenfolge.
- Du formulierst einen Untersuchungsauftrag mit klar begrenztem Umfang.
- Du trennst Beobachtungen von Schlussfolgerungen und erklärst, warum das für die Dokumentation zählt.

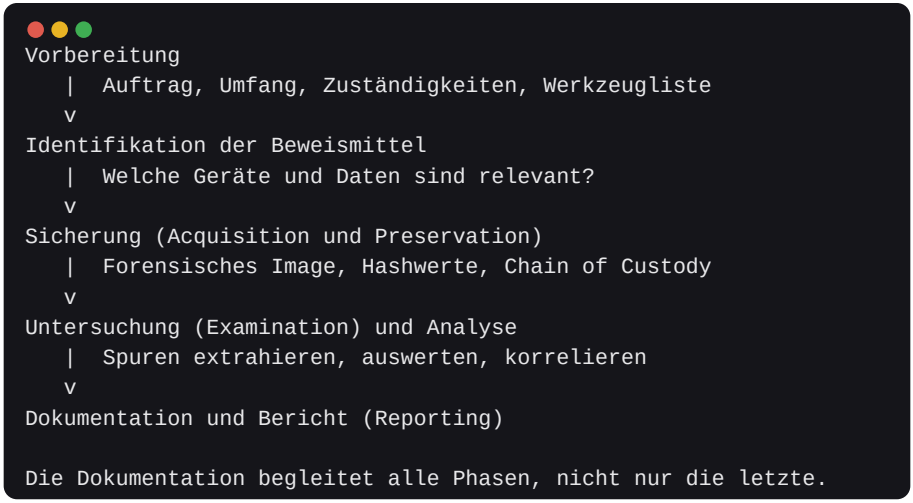
Ein Ausgangsbeispiel

Die Buchhaltung meldet, dass in den letzten Wochen ungewöhnlich große Datenmengen auf einen externen Speicherdienst hochgeladen wurden. Der Verdacht richtet sich auf den Laptop einer Mitarbeiterin aus dem Vertrieb. Dieses Beispiel begleitet das Kapitel. Es zeigt, wie aus einer vagen Beobachtung ein Untersuchungsauftrag wird und welche Schritte danach folgen.

Die Phasen im Überblick

Die internationale Norm ISO/IEC 27037 beschreibt, wie digitale Beweismittel identifiziert, gesammelt, gesichert und erhalten werden. NIST SP 800-86 gliedert den Ablauf einer forensischen Untersuchung in Sammlung, Untersuchung, Analyse und Berichterstattung. Beide Quellen ergänzen sich, weshalb dieses Buch die Phasen so ordnet:

Diagramm 2.1: Phasen einer forensischen Untersuchung



Die Pfeile machen den Ablauf sichtbar, sind aber nicht starr. Wer bei der Analyse auf Daten stößt, die im Umfang nicht vorgesehen waren, geht zurück zur Vorbereitung und erweitert den Auftrag schriftlich. Vorwärts zu hasten und dabei zurückzuspringen, ohne das zu dokumentieren, ist einer der häufigsten Fehler.

Vorbereitung und Untersuchungsauftrag

Bevor ein Gerät angefasst wird, braucht die Untersuchung einen schriftlichen Auftrag. Er beantwortet drei Fragen: Was soll herausgefunden werden? Welche Systeme und welcher Zeitraum sind betroffen? Wer entscheidet über Erweiterungen? Diese Grenze heißt **Scope**. Ein Auftrag wie „Prüfe den Laptop auf Auffälligkeiten“ ist zu weit, weil er keine Frage stellt, die sich beantworten lässt. Besser ist: „Ermittle, ob zwischen dem 1. und dem 30. des Monats Dateien aus dem Ordner Vertrieb an einen externen Dienst übertragen wurden, und wenn ja, welche.“

Zur Vorbereitung gehört außerdem die Werkzeugliste mit Versionen. Wer später nachvollziehen soll, wie ein Ergebnis entstanden ist, muss wissen, mit welcher Software und welchem Versionsstand gearbeitet wurde.

Identifikation der Beweismittel

Im Beispiel sind das zunächst der Laptop, sein Systemdatenträger und eventuell ein privates Smartphone, falls die Mitarbeiterin es für dienstliche Zwecke genutzt hat. Die Identifikation entscheidet, was überhaupt gesichert wird. Dabei gilt: Lieber ein Gerät zu viel auf der Liste als eines zu wenig, aber jede Aufnahme muss begründet sein. Ob ein privates Gerät überhaupt untersucht werden darf, ist eine rechtliche Frage, die vor der Sicherung geklärt werden muss, nicht danach. Ob und wie das Ergebnis rechtlich zu bewerten ist, entscheiden Auftraggeber und gegebenenfalls Juristen, nicht die Untersuchung allein.

Sicherung: Acquisition und Preservation

Die Sicherung besteht aus zwei Teilen. Bei der **Acquisition** wird eine vollständige, bitgenaue Kopie des Datenträgers erstellt, das forensische Image (Kapitel 5). Bei der **Preservation** wird dafür gesorgt, dass das Original unverändert bleibt und die Kopie seinen Zustand korrekt wiedergibt. Beides wird durch Hashwerte (Kapitel 6) und durch die Chain of Custody (Kapitel 3) belegt.

Die Sicherung ist der Punkt, an dem Fehler am wenigsten rückgängig zu machen sind. Wer den Laptop einschaltet, im Betriebssystem herumklickt oder Dateien öffnet, verändert Zeitstempel und legt Verlaufsdateien an. Solche Veränderungen sind später nicht mehr von den ursprünglichen Spuren zu unterscheiden.

Untersuchung und Analyse

Bei der **Examination** werden Daten aus dem Image extrahiert, etwa Dateien, Protokolle, Registry-Einträge oder Browserverläufe. Bei der **Analysis** werden diese Daten interpretiert, zueinander in Beziehung gesetzt und auf die Untersuchungsfrage bezogen. Der Unterschied ist wichtig: Eine Datei mit dem Namen „Kundenliste_final.xlsx“ im Verzeichnis eines USB-Speichers ist ein extrahierter Befund. Dass sie absichtlich kopiert wurde, ist eine Analyse, die weitere Belege braucht.

In Kapitel 8 zeigen wir, wie Zeitangaben aus verschiedenen Quellen zu einer Timeline zusammengeführt werden. Eine einzelne Quelle beweist selten etwas, mehrere unabhängige Quellen, die dasselbe zeigen, sind deutlich

belastbarer.

Dokumentation und Bericht

Der Bericht ist das Ergebnis der gesamten Arbeit. Er muss so geschrieben sein, dass eine Person, die den Fall nicht kennt, die Schritte nachvollziehen und zu demselben Befund kommen kann. Das nennt man **Reproduzierbarkeit**. Sie setzt voraus, dass jeder Schritt mit Zeitangabe, Werkzeug, Version und Ergebnis festgehalten ist. Den Aufbau eines Berichts übt das Praxisprojekt in Kapitel 10.

Beobachtung und Schlussfolgerung trennen

Die wichtigste Unterscheidung im Bericht ist die zwischen Beobachtung und Schlussfolgerung. Eine Beobachtung hält fest, was das Werkzeug ausgegeben hat: „Am 12. des Monats wurde die Datei angelegt, Zeitstempel laut Dateisystem 09:14 UTC.“ Eine Schlussfolgerung deutet das: „Die Datei wurde vermutlich vom Benutzerkonto der Mitarbeiterin angelegt.“ Beides kann richtig sein, aber nur das erste ist unmittelbar belegt. Schreibe deshalb Beobachtungen in der Form, in der sie gemessen wurden, und kennzeichne Schlussfolgerungen ausdrücklich als solche.

Umgang mit Fehlern

Auch erfahrene Untersucher machen Fehler: ein falsches Laufwerk, ein vergessenes Protokoll, ein Werkzeug, das eine Option anders interpretiert als erwartet. Entscheidend ist, dass Fehler nicht verschwiegen werden. Notiere sie im Protokoll mit Zeitpunkt und Auswirkung. Ein dokumentierter Fehler schadet dem Ergebnis weniger als ein verborgener, der später auffällt.

Typische Fehler

- Arbeit am Original statt an einer forensischen Kopie
- Untersuchung ohne schriftlichen Auftrag oder mit unklarem Scope
- Erweiterung des Umfangs, ohne den Auftrag anzupassen
- Schlussfolgerungen, bevor die Beobachtungen vollständig sind
- Fehlende Versionsangaben bei Werkzeugen
- Vermischung von Beobachtung und Deutung im Bericht

Zusammenfassung

- Eine forensische Untersuchung folgt den Phasen Vorbereitung, Identifikation, Sicherung, Untersuchung und Analyse sowie Dokumentation.
- Der schriftliche Auftrag begrenzt den Scope und bestimmt, welche Fragen beantwortet werden.
- Die Sicherung schützt das Original, weil Veränderungen später nicht mehr erkennbar sind.
- Beobachtungen werden getrennt von Schlussfolgerungen dokumentiert.

Übung A: Untersuchungsauftrag formulieren

Lernziel: einen Auftrag so formulieren, dass er beantwortbar und begrenzt ist. **Voraussetzungen:** keine technischen Voraussetzungen. **Aufgabenstellung:** Formuliere für das Ausgangsbeispiel einen Auftrag mit einer konkreten Untersuchungsfrage, einem Zeitraum, den betroffenen Geräten und einer Regel, wer über Erweiterungen entscheidet. **Erwartetes Ergebnis:** Ein Auftrag von höchstens fünf Sätzen, in dem jede Aussage prüfbar ist. **Überprüfung:** Kann man nach Abschluss der Untersuchung eindeutig sagen, ob die Frage beantwortet wurde? **Fehleranalyse:** Der Auftrag enthält Wörter wie „auffällig“ oder „verdächtig“ ohne Definition? Ersetze sie durch überprüfbare Kriterien. **Musterlösung:** Ein Auftrag, der Zeitraum, Gerät, Frage und Entscheidungsweg nennt, etwa: „Ermittle, ob im Zeitraum vom 1. bis 30. des Monats Dateien aus dem Ordner Vertrieb auf den Laptop-Systemdatenträger übertragen und an externe Speicherdienste hochgeladen wurden. Erweiterungen auf weitere Geräte entscheidet die Leitung der IT-Sicherheit schriftlich.“

Übung B: Beobachtung oder Schlussfolgerung?

Lernziel: die Unterscheidung im Berichtstext sicher anwenden. **Aufgabenstellung:** Ordne die folgenden Aussagen den Kategorien Beobachtung oder Schlussfolgerung zu: (a) Die Datei wurde um 14:02 Uhr geöffnet. (b) Die Mitarbeiterin wollte die Daten mitnehmen. (c) Das Protokoll enthält 37 Verbindungen zu einer bestimmten Adresse. (d) Die Verbindungen erfolgten vermutlich über einen Browser. **Erwartetes Ergebnis:** (a) und (c) sind Beobachtungen, (b) ist eine Schlussfolgerung über innere Absichten und nicht belegbar, (d) ist eine Schlussfolgerung, die sich aus weiteren Daten ergeben muss. **Erklärung:** Bei (a) und (c) steht im Protokoll oder Werkzeug, was gemessen wurde. Bei (b) wird eine Absicht unterstellt, die aus Daten allein nicht ablesbar ist.

Kontrollfragen

1. Welche Phasen umfasst eine forensische Untersuchung nach dem Ablauf dieses Kapitels? (leicht)
2. Was ist der Scope einer Untersuchung? (leicht)
3. Was unterscheidet Acquisition von Preservation? (mittel)
4. Warum ist ein Protokoll mit Werkzeugversionen für die Reproduzierbarkeit wichtig? (mittel)
5. Im Bericht steht: „Der Benutzer hat die Datei bewusst gelöscht.“ Was fehlt dieser Aussage, und wie formulierst du sie korrekt? (schwer)

Lösungshinweise: 1. Vorbereitung, Identifikation, Sicherung, Untersuchung und Analyse, Dokumentation. 2. Die Begrenzung der Untersuchung nach Systemen, Zeitraum und Fragestellung, festgelegt im schriftlichen Auftrag. 3. Acquisition erstellt die Kopie, Preservation sorgt dafür, dass Original und Kopie unverändert bleiben und belegt werden. 4. Nur mit Werkzeug und Version lässt sich ein Ergebnis von einer anderen Person mit denselben Daten nachvollziehen. 5. Die Aussage vermischt Beobachtung und Absicht. Korrekt wäre etwa: „Im Protokoll ist ein Löschvorgang für die Datei um 14:02 Uhr verzeichnet. Ob er bewusst erfolgte, lässt sich aus den Daten nicht sicher ableiten.“

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] die Phasen der Untersuchung in der richtigen Reihenfolge nennen kannst,
- [ ] einen Auftrag mit klar begrenztem Scope formulieren kannst,
- [ ] Beobachtungen und Schlussfolgerungen in einem Text auseinanderhalten kannst.

Kapitel 3: Beweissicherung und Chain of Custody

Lernziele

- Du erfasst ein digitales Beweismittel so, dass Identität, Zustand und Herkunft dokumentiert sind.
- Du erklärst den Zweck der Chain of Custody und führst ein Protokoll über alle Übergaben.
- Du erkennst typische Fehlerquellen bei der Beweissicherung und vermeidest sie.

Was ist ein Beweismittel, und was macht es belastbar?

Im vorigen Kapitel haben wir den Scope festgelegt. Jetzt geht es um die physischen Geräte, auf denen die Daten liegen. Ein digitales Beweismittel ist ein Datenträger oder Gerät mit Inhalten, die zum Auftrag gehören. Ob seine Ergebnisse später belastbar sind, hängt davon ab, wie gut dokumentiert ist, woher das Gerät stammt, in welchem Zustand es übernommen wurde und wer es zu welchem Zeitpunkt in der Hand hatte. Die Beweissicherung ist deshalb zuerst eine Dokumentationsaufgabe.

Identifikation

Jedes Gerät bekommt eine eindeutige Beschreibung: Art (Laptop, USB-Stick, Festplatte), Hersteller, Modell, Seriennummer, Kapazität und die äußeren Merkmale wie Gehäusefarbe, Aufkleber oder Schäden. Die Seriennummer findest du meist auf dem Gehäuse oder über das Gerät selbst. Notiere sie, bevor du das Gerät öffnest oder anschließt, denn spätere Gehäuseaufkleber lassen sich nicht immer eindeutig zuordnen.

Zustand dokumentieren

Dokumentiere den Zustand bei der Übernahme so genau, dass er später rekonstruierbar ist. Dazu gehören, ob das Gerät eingeschaltet, ausgeschaltet oder im Ruhezustand war, ob es angeschlossen war, welche Kabel daran hingen, und ob Siegel oder Gehäuseschrauben unbeschädigt waren. Fotografiere die Vorderseite, die Rückseite, Anschlüsse und das Typenschild. Die Fotos ersetzen keine Beschreibung, sie ergänzen sie.

Zeitangaben

Jede Notiz braucht eine Uhrzeit, und hier entstehen häufig Fehler. Die Uhr des Erfassungsgeräts kann von der Uhr des beschlagnahmten Geräts abweichen, und Zeitzonen führen leicht zu Verwechslungen. Notiere deshalb immer Datum, Uhrzeit und Zeitzone (am besten UTC) und halte fest, wie die Uhrzeit ermittelt wurde. Die Systemzeit des untersuchten Geräts ist ein eigenes Thema, das Kapitel 8 vertieft. Hier geht es nur darum, dass die Zeit deiner Notizen unabhängig von diesem Gerät nachvollziehbar ist.

Beweismittellabel und Nummerierung

Jedes Beweismittel bekommt eine eindeutige Nummer nach einem festen Schema, etwa Jahr, Fallnummer und laufende Zahl. Die Nummer steht auf einem Etikett am Gerät und in allen Protokollen. Das Etikett sollte so angebracht sein, dass es sich beim Öffnen einer Tüte nicht unbemerkt ablösen lässt. Ein Etikett ohne Nummer im Protokoll oder eine Nummer ohne Etikett am Gerät sind beide Fehler, die sich später nur schwer beheben lassen.

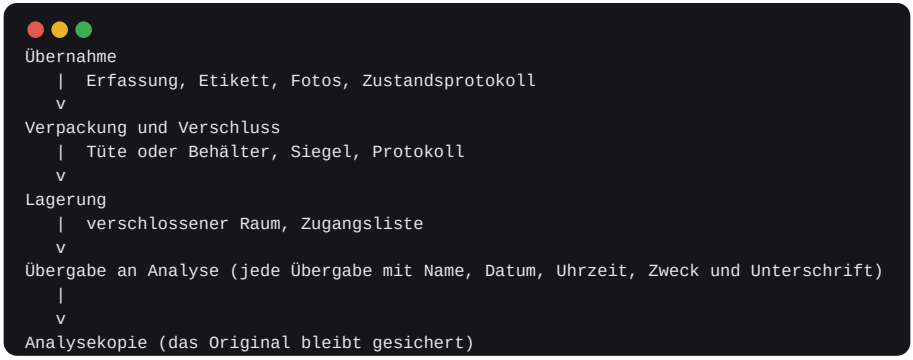
Verantwortliche Personen

Im Protokoll steht, wer das Gerät übernommen, bearbeitet und weitergegeben hat. Jede Person wird mit vollständigem Namen, Funktion und Unterschrift erfasst. Wer ein Gerät nur kurz anfasst, zählt ebenfalls, denn auch dieser Kontakt gehört zur Kette.

Chain of Custody

Die **Chain of Custody** (Beweismittelkette) ist die lückenlose Dokumentation, wer ein Beweismittel wann in Besitz hatte, wo es lag und zu welchem Zweck es übergeben wurde. Sie beantwortet nicht, ob das Beweismittel echt ist. Sie zeigt aber, ob es in der Zwischenzeit unbeobachtet war. Eine Lücke in der Kette wirft Fragen auf, auch wenn nichts verändert wurde.

Diagramm 3.1: Ablauf der Chain of Custody



Manipulationsschutz und Lagerung

Verpacke Geräte so, dass Veränderungen sichtbar werden. Siegel auf Gehäuse und Tüten zeigen, ob jemand sie geöffnet hat. Lagere Geräte trocken, vor starken Temperaturschwankungen und vor statischer Entladung geschützt, und nur in einem Raum, zu dem eine Zugangsliste geführt wird. Die Lagerung ist Teil der Beweissicherung, auch wenn sie unspektakulär wirkt.

Schreibschutz

Der wichtigste technische Schutz ist der Schreibschutz. Ein **Write-Blocker** ist ein Hardwaregerät oder eine Schnittstelle, die Schreibzugriffe auf den angeschlossenen Datenträger unterdrückt, während Lesezugriffe erlaubt bleiben. Er liegt zwischen Untersuchungsrechner und Datenträger. Ein Software-Schreibschutz im Betriebssystem ist dagegen nicht gleichwertig: Er kann umgangen werden, wirkt nicht auf jede Schicht und hängt von Einstellungen ab, die du vielleicht nicht vollständig überprüfst. Ein Write-Blocker verringert das Risiko, ersetzt aber nicht die Dokumentation.

Auch ohne Schreibzugriff kann das Betriebssystem beim Anschließen Dinge tun, etwa Partitionen einhängen oder Zeitstempel anfassen, sofern dies nicht verhindert wird. Deshalb gehört die Frage „Was passiert, wenn ich das anschließe?“ vor jeden Anschluss.

Physische und logische Sicherung

Bei der **physischen Sicherung** wird der gesamte Datenträger, also alle Sektoren, unabhängig von Dateisystem und Partitionen kopiert. Bei der **logischen Sicherung** werden nur Dateien und Verzeichnisse kopiert, etwa mit einem Dateimanager. Die logische Kopie ist für eine forensische Untersuchung in der Regel unvollständig, weil gelöschte Dateien, Restbereiche und Metadaten fehlen. Kapitel 5 behandelt die physische Sicherung ausführlich. Die logische Kopie hat trotzdem ihren Platz, etwa wenn ein Auftrag nur bestimmte Ordner betrifft und dies schriftlich festgelegt ist.

Umgang mit beschädigten Datenträgern

Ein beschädigter Datenträger, der klicken, nicht erkannt werden oder Wärme entwickeln kann, sollte nicht selbst geöffnet oder repariert werden. Jeder Eingriff kann die Daten endgültig verändern oder zerstören. Dokumentiere den Zustand, sichere die Umgebung und hole gegebenenfalls ein spezialisiertes Labor hinzu. Diese Entscheidung ist ebenfalls zu protokollieren.

Typische Fehlerquellen

- Gerät eingeschaltet übernommen, ohne den Zustand zu dokumentieren
- Anschluss an den Arbeitsrechner statt an einen Write-Blocker
- Zeitangaben ohne Zeitzone oder mit unklarer Uhr
- Lücken in der Chain of Custody, etwa ein Gerät ohne Protokoll über Nacht
- Etiketten, die nicht eindeutig zur Nummer im Protokoll passen
- Verlassen auf den Software-Schreibschutz als einzige Schutzmaßnahme

> **WARNUNG – Sicherheitsrisiko**
>
> Schließe ein Beweismittel niemals an ein System an, auf dem du dich mit Schreibrechten bewegst, ohne vorher einen Schreibschutz eingerichtet und geprüft zu haben. Ein einziger Anschluss kann Zeitstempel und Verzeichnisse verändern, bevor du merkst, dass etwas schiefläuft.

Praxis: Beweismittel erfassen

Übung A: Beweismittel mit Formular erfassen

Lernziel: ein Gerät vollständig und nachvollziehbar erfassen. **Voraussetzungen:** ein eigenes Testgerät, etwa ein USB-Stick, und ein Blatt oder eine Datei für das Protokoll. **Aufgabenstellung:** Erfasse einen eigenen Testdatenträger mit der folgenden Vorlage. Die Einträge in der rechten Spalte sind fiktive Beispiele und zeigen, wie ein ausgefüllter Eintrag aussehen kann.

Feld	Eintrag (fiktives Beispiel)
Beweismittelnummer	BM-2026-0001
Gerätetyp	USB-Speicherstick
Hersteller und Modell	Beispielhersteller, Modell X
Seriennummer	Vom Gerät abgelesen, z. B. 4C530001
Kapazität	64 GB
Zustand bei Übernahme	Nicht angeschlossen, Gehäuse unbeschädigt
Datum und Uhrzeit (UTC)	2026-11-10, 14:20 UTC
Uhr und Quelle der Zeit	Erfassungsrechner, Zeit per NTP-Synchronisation
Übernommen von	Name, Funktion, Unterschrift
Zweck der Übernahme	Übung Beweissicherung

Feld	Eintrag (fiktives Beispiel)
Verschlussnummer	Siegel SL-0001

Erwartetes Ergebnis: Ein vollständig ausgefülltes Protokoll mit eindeutiger Nummer, Zustandsbeschreibung und Zeitangabe. **Überprüfung:** Kann eine dritte Person anhand des Protokolls das Gerät eindeutig wiederfinden und den Zustand bei der Übernahme nachvollziehen? **Fehleranalyse:** Fehlt eine Zeitzone? Ergänze sie und notiere, woher die Uhrzeit stammt.

Übung B: Beweismittelnummer nach festem Schema vergeben

Lernziel: ein Nummernschema anwenden, das sich nicht versehentlich doppelt. **Aufgabenstellung:** Lege ein Schema fest, das Jahr, Fallnummer und laufende Nummer kombiniert, und vergib die Nummern für drei fiktive Geräte eines Falls. **Erwartetes Ergebnis:** Drei eindeutige Nummern, etwa BM-2026-0001 bis BM-2026-0003, die sich aus dem Schema ableiten lassen. **Fehleranalyse:** Zwei Geräte bekommen dieselbe Nummer? Prüfe, ob die laufende Nummer im Protokoll fortgeführt wurde.

Übung C: Gerät identifizieren und Schreibschutz prüfen

Lernziel: Datenträger mit Lesebefehlen eindeutig identifizieren und den Schreibschutzstatus ablesen. **Voraussetzungen:** Linux-VM mit einem angeschlossenen Testdatenträger, zum Beispiel einem USB-Stick aus Übung A. **Aufgabenstellung:** Ermittle den Gerätenamen des Testdatenträgers und prüfe, ob er als schreibgeschützt gemeldet wird.

Befehle zu Übung C

```
$ lsblk -o NAME,SIZE,MODEL,SERIAL,TRAN,RO
$ sudo blockdev --getro /dev/sdX
```

Erklärung: `lsblk` listet angeschlossene Datenträger mit Größe, Modell, Seriennummer, Übertragungsweg und dem Schreibschutzstatus (Spalte RO, 1 bedeutet schreibgeschützt). `blockdev --getro` gibt den vom Linux-Kernel gemeldeten Read-only-Status des Geräts als Zahl aus, 1 für schreibgeschützt und 0 für beschreibbar. Dieser Wert ist allein kein unabhängiger Nachweis dafür, dass ein vorgeschalteter Hardware-Write-Blocker korrekt arbeitet. Die Funktion eines Write-Blockers ist nach den Herstellerangaben zu prüfen oder mit einem geeigneten

Verfahren zu verifizieren. Ersetze `/dev/sdX` durch den Namen aus der ersten Ausgabe, und wähle ihn anhand von Größe und Modell aus, nicht nach Gefühl. **Erwartetes Ergebnis:** Der Testdatenträger erscheint mit Größe und Seriennummer, und die Ausgabe von `blockdev` zeigt, ob er schreibgeschützt ist. Mit einem Write-Blocker gilt dieser Status nur, wenn der Write-Blocker selbst funktioniert, was du gesondert verifizierst. Ohne ihn ist der Status auf dem Untersuchungsrechner eine Einstellung des Systems, die sich ändern kann. **Typische Fehler:** Das falsche Gerät wird ausgewählt, etwa die Systemfestplatte des Untersuchungsrechners. Ein Befehl mit `dd` oder ähnlichen Schreibwerkzeugen gegen den falschen Namen zerstört dort Daten. In dieser Übung werden ausschließlich Lesebefehle verwendet. **Troubleshooting:** Der Datenträger fehlt in der Ausgabe? Prüfe mit `dmesg | tail` die Erkennung beim Anschließen. **Sicherheitsbetrachtung:** Verwende nur Testdatenträger, die du selbst angeschlossen hast. Vorab prüfen, ob der Name zum erwarteten Gerät passt.

Kontrollfragen

1. Wozu dient die Seriennummer im Erfassungsprotokoll? (leicht)
2. Was ist ein Write-Blocker? (leicht)
3. Warum reicht eine Zeitangabe ohne Zeitzone nicht aus? (mittel)
4. Ein Protokoll zeigt, dass ein Gerät über Nacht ohne Eintrag im Lagerraum lag. Was bedeutet das für die Chain of Custody? (mittel)
5. Ein Ermittler schließt das Beweismittel an seinen Arbeitsrechner an, aktiviert dort den Software-Schreibschutz und beginnt die Analyse. Welche Risiken bleiben, und welche Maßnahmen wären besser gewesen? (schwer)

Lösungshinweise: 1. Sie identifiziert das Gerät eindeutig, auch wenn Etiketten sich lösen. 2. Ein Hardwaregerät oder eine Schnittstelle, die Schreibzugriffe auf den Datenträger unterdrückt, während Lesezugriffe möglich bleiben. 3. Ohne Zeitzone bleibt unklar, auf welche Uhr sich die Angabe bezieht, und Zeitvergleiche mit anderen Quellen werden unzuverlässig. 4. Die Kette ist an dieser Stelle unterbrochen, was Fragen zur Unversehrtheit aufwirft. Die Lücke muss im Protokoll erklärt und gegebenenfalls bewertet werden. 5. Der Software-Schreibschutz kann umgangen werden und wirkt nicht auf jede Schicht, zudem können beim

Anschluss Zeitstempel und Einhängevorgänge entstehen. Besser wären ein Write-Blocker, eine vorherige Dokumentation des Zustands und eine Analyse auf einer forensischen Kopie.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] ein Beweismittel vollständig und mit eindeutiger Nummer erfassen kannst,
- [ ] die Chain of Custody erklären und eine Übergabe korrekt protokollieren kannst,
- [ ] den Unterschied zwischen Software-Schreibschutz und Write-Blocker erklären kannst,
- [ ] einen Testdatenträger mit Lesebefehlen identifizieren kannst, ohne versehentlich zu schreiben.

> **Zusammenfassung**
>
> • Jedes Beweismittel wird eindeutig identifiziert, mit Zustand, Zeitangabe mit Zeitzone und Nummer dokumentiert.
> • Die Chain of Custody hält lückenlos fest, wer das Gerät wann hatte und warum es übergeben wurde.
> • Ein Write-Blocker schützt den Datenträger zuverlässiger als ein Software-Schreibschutz, ersetzt aber nicht die Dokumentation.
> • Die Identifikation des richtigen Datenträgers vor jedem Befehl ist Pflicht.

Kapitel 4: Forensische Datenträgerimages

Lernziele

- Du erklärst den Unterschied zwischen einem Backup und einem forensischen Image.
- Du erstellst eine Bit-für-Bit-Kopie und prüfst sie byte- und hashgenau gegen das Original.
- Du unterscheidest die Partitionsstrukturen MBR und GPT und weißt, welche Bereiche ein Image enthalten muss.

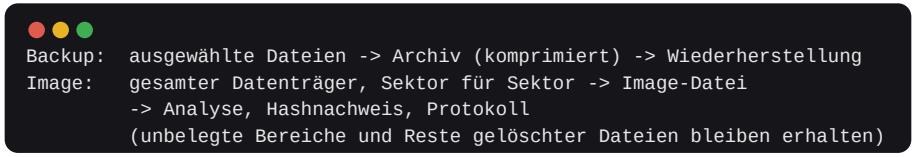
Was ist ein forensisches Image?

Ein forensisches Image ist eine vollständige, bitgenaue Kopie eines Datenträgers. Jeder Sektor wird übernommen, auch die Bereiche, die das Dateisystem als frei markiert. Dadurch bleiben Reste gelöschter Dateien erhalten, ebenso Metadaten, die eine normale Dateikopie verlieren würde. Ein Image zeigt damit, wie der Datenträger zum Zeitpunkt der Sicherung aussah, nicht nur, welche Dateien darauf sichtbar waren.

Backup und Image

Ein Backup soll Daten wiederherstellbar machen. Es kopiert ausgewählte Dateien, oft komprimiert, und lässt unbelegte Bereiche weg. Ein Image soll Beweise sichern. Es kopiert den Datenträger als Ganzes, ohne Auswahl und ohne Umformatierung. Der Zweck bestimmt die Form: Ein Backup beantwortet die Frage, ob sich ein Zustand wiederherstellen lässt. Ein Image beantwortet die Frage, wie der Datenträger aussah und ob sich das belegen lässt.

Diagramm 4.1: Backup gegenüber forensischem Image



Bit-für-Bit-Kopie

Bitgenau bedeutet: Jedes Bit im Image steht an derselben Stelle wie auf dem Original. Ein Werkzeug, das nach Dateien sortiert kopiert, erfüllt diese Bedingung nicht. Das Kommandozeilenwerkzeug `dd` liest dagegen Blöcke in ihrer Reihenfolge und schreibt sie unverändert weiter. Deshalb ist es für diesen Zweck geeignet, sofern der Aufruf stimmt. Die Kontrolle durch Hashwerte und Vergleich folgt in Kapitel 6.

Physische und logische Sicherung

Kapitel 3 hat den Unterschied bereits beschrieben. Die physische Sicherung kopiert den gesamten Datenträger, die logische nur Dateien und Verzeichnisse. Eine forensische Untersuchung braucht die physische Sicherung, weil nur sie die unbelegten Bereiche und die Strukturen darunter enthält.

Datenträgerstruktur: MBR und GPT

Ein Datenträger ist in Partitionen gegliedert, und jede Partition trägt ein Dateisystem. Die Aufteilung steht in der Partitionstabelle. Zwei Varianten sind verbreitet.

Das **MBR** (Master Boot Record) steht im ersten Sektor des Datenträgers. Es erlaubt bis zu vier primäre Partitionen und Datenträger bis etwa 2 TiB bei 512-Byte-Sektoren.

Das **GPT** (GUID Partition Table) nutzt einen größeren Kopf am Anfang und eine Sicherungskopie am Ende des Datenträgers. Es unterstützt deutlich mehr Partitionen und sehr große Datenträger.

Diagramm 4.2: Aufbau eines Datenträgers (vereinfacht)



Für das Image heißt das: Ein Abbild, das nur den Anfang des Datenträgers umfasst, ist unvollständig. Bei GPT gehört die Sicherungskopie am Ende ebenso dazu wie alle Partitionen.

Unbelegter Speicher und Slack Space

Der **unbelegte Speicher** (unallocated space) umfasst Bereiche, die das Dateisystem als frei markiert. Hier können Reste gelöschter Dateien liegen, bis sie überschrieben werden. Der **Slack Space** ist der Rest eines Clusters oder Blocks, den eine Datei nicht ganz ausfüllt. Auch dort kann Inhalt früherer Dateien stehen. Beide Bereiche sind nur im physischen Image sichtbar.

Vorsicht bei der Auswertung: Dass ein Rest gefunden wurde, zeigt nicht, zu welcher Datei er gehörte. Eine solche Zuordnung braucht zusätzliche Belege und wird im Bericht als Interpretation gekennzeichnet (Kapitel 2).

Schreibschutz

Vor dem Anschluss eines Datenträgers an den Untersuchungsrechner muss der Schreibschutz stehen. Ein Write-Blocker, wie in Kapitel 3 beschrieben, verhindert Schreibzugriffe. Das Original wird nie gestartet, nie eingehängt und nie zum Testen beschrieben.

Imageformate

RAW ist eine reine Bitkopie ohne zusätzliche Informationen. Das Format ist einfach und von vielen Werkzeugen lesbar. Es enthält aber keine Metadaten, Hashwerte, Zeitangaben oder Angaben zum Ersteller, weshalb diese Informationen separat festgehalten werden müssen.

E01 (Expert Witness Format) speichert die Bitkopie zusammen mit Metadaten und Prüfsummen in einer Datei, die sich komprimieren und in Segmente teilen lässt. Es ist in der forensischen Praxis weit verbreitet.

AFF4 (Advanced Forensic Format 4) ist ein offenes, erweiterbares Format mit Metadaten. Es ist weniger verbreitet als E01, aber dokumentiert und für eigene Auswertungen gut zugänglich.

Die Erstellung eines E01- oder AFF4-Images erfolgt mit forensischen Werkzeugen, deren Bedienung in deren Dokumentation beschrieben ist. Dieses Kapitel zeigt die Grundlagen mit `dd` und RAW, weil die Befehle überschaubar sind und sich mit einfachen Mitteln prüfen lassen.

Metadaten und Protokoll

Zu jedem Image gehören Angaben, die außerhalb der Bitkopie festgehalten werden: Quelle und Seriennummer, Zeitpunkt der Erstellung samt Zeitzone, verwendetes Werkzeug mit Version, Befehl oder Einstellungen, Hashwert vor und nach der Erstellung, sowie Lesefehler, falls welche auftraten. Bei RAW-Images bildet dieses Protokoll die einzige Dokumentation.

Integritätsprüfung

Der Hashwert des Originals wird vor der Sicherung berechnet, der des Images danach. Stimmen beide überein, ist das Image eine vollständige Kopie der Daten zum Zeitpunkt der Sicherung. Kapitel 6 behandelt den Vergleich und seine Grenzen ausführlich.

Typische Fehler

- `if` und `of` vertauscht, wodurch das Original überschrieben wird
- Ziel zu klein oder ohne freien Speicherplatz, sodass das Image unvollständig bleibt
- Ausgelassene Lesefehler im Protokoll, obwohl `dd` Lücken mit Nullen auffüllt
- Kein Hashwert vor oder nach der Erstellung
- Imagedatei auf demselben Datenträger wie das Original gespeichert

WARNUNG – Sicherheitsrisiko

Bei `dd` entscheidet die Reihenfolge der Argumente über Erfolg oder Zerstörung. `if=` bezeichnet die Quelle, `of=` das Ziel. Vertauschst du beide, schreibt `dd` ohne Rückfrage auf das Original. Prüfe jeden Befehl vor dem Start, insbesondere bei Datenträgern mit Daten, die du behalten willst.

Gut zu wissen

Die Option `conv=noerror,sync` führt dazu, dass `dd` bei Lesefehlern weiterarbeitet und die betroffenen Blöcke mit Nullen auffüllt. Das Image ist dann an diesen Stellen nicht identisch mit dem Original. Die Lücken müssen im Protokoll mit Position und Anzahl der betroffenen Blöcke vermerkt werden. Bei fehlerhaften Datenträgern ist ein spezialisiertes Werkzeug wie GNU ddrescue vorzuziehen, das Lesefehler gezielt wiederholt.

Praxis: Testdatenträger, Image und Prüfung

Die folgenden Übungen verwenden eine Datei als Stellvertreter für einen Datenträger. So kannst du jeden Schritt wiederholen, ohne ein echtes Gerät zu gefährden. Verwende für Übung C einen eigenen Testdatenträger, den du selbst eingerichtet hast.

Übung A: Testdatenträger erzeugen

Lernziel: einen reproduzierbaren Testdatenträger mit bekanntem Hashwert anlegen. **Voraussetzungen:** Linux-VM, Schreibrechte im Arbeitsverzeichnis. **Aufgabenstellung:** Erzeuge eine 64 MiB große Datei mit Zufallsdaten und berechne ihren Hashwert.

Befehle zu Übung A

```
$ dd if=/dev/urandom of=testdisk.img bs=1M count=64
    status=progress
$ sha256sum testdisk.img
```

Erklärung: `if` ist die Quelle, hier der Zufallsgenerator des Systems. `of` ist das Ziel. `bs=1M` und `count=64` ergeben zusammen 64 MiB. `status=progress` zeigt den Fortschritt an, wie es die GNU-Version von `dd` unterstützt. **Erwartetes Ergebnis:** Eine Datei `testdisk.img` mit einer

Größe von 67108864 Byte und einem 64-stelligen Hashwert. Der Hashwert ändert sich bei jeder Erzeugung, weil die Zufallsdaten neu sind. Notiere ihn. **Überprüfung:** `ls -l testdisk.img` zeigt die erwartete Größe. **Fehleranalyse:** Die Datei fehlt? Prüfe, ob du im richtigen Verzeichnis arbeitest.

Übung B: Bit-für-Bit-Image erstellen und vergleichen

Lernziel: ein Image erzeugen und seine Identität auf zwei Wegen belegen. **Voraussetzungen:** Übung A abgeschlossen. **Aufgabenstellung:** Kopiere `testdisk.img` in das Image `beweis.raw` und vergleiche Original und Kopie.

Befehle zu Übung B

```
$ dd if=testdisk.img of=beweis.raw bs=4M conv=noerror,sync
    status=progress
$ sha256sum testdisk.img beweis.raw
$ cmp testdisk.img beweis.raw && echo "identisch"
```

Erklärung: `cmp` vergleicht beide Dateien Byte für Byte. Eine leere Ausgabe mit dem Text „identisch“ bedeutet, dass kein Unterschied gefunden wurde. Die beiden Hashwerte müssen ebenfalls gleich sein. Das Zusammenspiel beider Prüfungen ist wichtig, weil ein Hashwert nur so gut ist wie die Berechnung, und `cmp` zeigt die Gleichheit unabhängig davon. **Erwartetes Ergebnis:** Beide Hashwerte sind identisch, `cmp` meldet keinen Unterschied. **Überprüfung:** Trage Hashwert, Zeitpunkt und Befehl in ein kurzes Protokoll ein. **Fehleranalyse:** Die Hashwerte unterscheiden sich? Prüfe, ob `if` und `of` richtig gesetzt sind und ob die Kopie vollständig geschrieben wurde (Fehlermeldungen von `dd` beachten).

Übung C: Partitionsstruktur eines Images lesen

Lernziel: die Partitionstabelle eines Images untersuchen, ohne es einzuhängen. **Voraussetzungen:** ein eigener Testdatenträger mit einer Partitionstabelle, zum Beispiel ein USB-Stick, den du selbst partitioniert hast. Der Stick muss mit Schreibschutz oder über einen Write-Blocker angeschlossen sein. **Aufgabenstellung:** Erstelle ein RAW-Image des Testdatenträgers, hänge das Image schreibgeschützt als Schleifengerät ein und lies die Partitionstabelle aus.

Befehle zu Übung C

```
$ sudo dd if=/dev/sdX of=stick.raw bs=4M conv=noerror,sync
    status=progress
$ sudo losetup -r -f --show stick.raw
$ sudo fdisk -l /dev/loopN
$ sudo losetup -d /dev/loopN
```

Erklärung: Die erste Zeile erstellt das Image. `losetup -r` hängt die Datei schreibgeschützt als Blockgerät ein, `-f --show` wählt das erste freie Schleifengerät und gibt seinen Namen aus. `fdisk -l` zeigt die Partitionstabelle an, ohne Änderungen vorzunehmen. `losetup -d` löst die Einhängung wieder. Ersetze `/dev/sdX` durch den Namen des Testdatenträgers, den du vorher anhand von Größe und Modell mit `lsblk` identifiziert hast, und `/dev/loopN` durch den Namen aus der Ausgabe von `losetup`. **Erwartetes Ergebnis:** `fdisk -l` zeigt Typ der Partitionstabelle (dos für MBR, gpt für GPT) sowie Start- und Endsektoren aller Partitionen. **Überprüfung:** Stimmt die Partitionsgröße mit dem Datenträger überein, und entspricht die Anzahl der Partitionen deiner Einrichtung? **Fehleranalyse:** `fdisk` zeigt keine Partitionen? Der Datenträger hat möglicherweise keine Tabelle. Das ist selbst ein Befund und gehört ins Protokoll. Das Gerät wurde nicht korrekt eingehängt? Prüfe die Ausgabe von `losetup -a`. **Sicherheitsbetrachtung:** Der Befehl `dd` liest hier vom Stick. Vertausche `if` und `of` niemals, und verwende ausschließlich den Testdatenträger.

Übung D: Image sichern und protokollieren

Lernziel: das Image mit Hashwert und Protokoll so ablegen, dass es später prüfbar bleibt. **Aufgabenstellung:** Lege für `beweis.raw` eine Hashdatei an und ein Protokoll mit Metadaten.

Befehle zu Übung D

```
$ sha256sum beweis.raw > beweis.raw.sha256
$ sha256sum -c beweis.raw.sha256
```

Erklärung: Die erste Zeile speichert den Hashwert in einer Datei. Die zweite Zeile prüft das Image erneut gegen diesen Wert. Die Ausgabe `beweis.raw: OK` bestätigt die Übereinstimmung. Das Protokoll enthält zusätzlich die Angaben aus dem Abschnitt Metadaten dieses Kapitels. **Erwartetes Ergebnis:** `OK` in der Ausgabe und ein Protokoll, das Quelle,

Zeitpunkt, Werkzeug und Hashwert nennt. **Fehleranalyse:** `FEHLGESCHLAGEN` bedeutet, dass sich die Datei seit der Hashberechnung verändert hat. Prüfe, ob das Image verschoben, kopiert oder anders gespeichert wurde.

Kontrollfragen

1. Worin unterscheidet sich ein Backup von einem forensischen Image? (leicht)
2. Warum enthält ein Image auch Bereiche, die das Dateisystem als frei markiert? (leicht)
3. Warum reicht ein Image, das nur den Anfang eines GPT-Datenträgers umfasst, nicht aus? (mittel)
4. Welche Bedeutung hat `conv=noerror,sync` für die Aussagekraft des Images? (mittel)
5. Ein RAW-Image wird ohne Protokoll und ohne Hashwert archiviert, weil „die Kopie ja vollständig aussieht“. Welche Probleme entstehen, und welche Angaben fehlen? (schwer)

Lösungshinweise: 1. Ein Backup kopiert ausgewählte Dateien für die Wiederherstellung, ein Image den gesamten Datenträger für die Beweissicherung. 2. Weil dort Reste gelöschter Dateien liegen können, die für die Untersuchung wichtig sind. 3. Die Sicherungskopie der GPT-Struktur am Ende des Datenträgers gehört zur vollständigen Struktur und fehlt sonst. 4. Lesefehler werden mit Nullen aufgefüllt, das Image ist an diesen Stellen nicht identisch mit dem Original, und die Lücken müssen dokumentiert werden. 5. Ein visueller Eindruck ersetzt keinen Nachweis: Ohne Hashwert ist die Übereinstimmung mit dem Original nicht belegbar, ohne Protokoll fehlen Quelle, Zeitpunkt, Werkzeug und Lesefehler, und das RAW-Format enthält diese Angaben nicht selbst.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Backup und forensisches Image sicher voneinander abgrenzen kannst,

[ ] ein Image erstellt und seine Identität mit `cmp` und `sha256sum` belegt hast,

[ ] MBR und GPT unterscheiden und die Partitionstabelle eines Images lesen kannst,

[ ] erklären kannst, welche Angaben zu jedem Image gehören.

> **Zusammenfassung**
>
> • Ein forensisches Image ist eine bitgenaue Kopie des gesamten Datenträgers, einschließlich unbelegter Bereiche.
> • Ein Image braucht ein Protokoll mit Quelle, Zeitpunkt, Werkzeug und Lesefehlern sowie Hashwerte vor und nach der Erstellung.
> • MBR und GPT unterscheiden sich im Aufbau, die Sicherungskopie des GPT am Ende gehört zum vollständigen Image.
> • Bei `dd` entscheidet die Reihenfolge von `if` und `of` über Erfolg oder Zerstörung.

Kapitel 5: Hashwerte und Integritätsprüfung

Lernziele

- Du erklärst, was ein Hashwert leistet und wo seine Grenzen liegen.
- Du berechnest und vergleichst Hashwerte unter Linux und Windows und prüfst eine Datei gegen eine Hashliste.
- Du dokumentierst Prüfergebnisse so, dass ihr Zeitpunkt und ihr Umfang eindeutig sind.

Was ein Hashwert ist

Ein kryptographischer Hashwert ist ein Fingerabdruck fester Länge, der aus beliebig vielen Daten berechnet wird. Ändert sich eine einzige Stelle der Eingabe, ändert sich der Hashwert vollständig. Dieser Effekt macht Hashwerte für die digitale Forensik nützlich: Man kann zwei Datenbestände vergleichen, ohne sie Byte für Byte nebeneinanderzulegen.

Hashwerte sind außerdem deterministisch. Dieselben Daten ergeben mit demselben Verfahren immer denselben Wert. Genau deshalb lässt sich ein Hashwert, der heute berechnet wird, mit einem Wert vergleichen, der vor einem Jahr notiert wurde.

Verfahren und ihr Status

MD5 und **SHA-1** sind ältere Verfahren, die in alten Berichten, Werkzeugen und Archiven noch vorkommen. Für Kollisionen, also zwei verschiedene Eingaben mit demselben Hashwert, sind beide inzwischen praktisch erzeugbar. Für die Integritätsprüfung in neuen Untersuchungen sind sie daher nicht geeignet, wenn es um Manipulation durch eine gezielt handelnde Person geht. Ein Vergleich mit einem alten MD5-Wert aus einer früheren Dokumentation bleibt trotzdem sinnvoll, sofern beide Seiten dasselbe Verfahren nutzen.

SHA-256 und **SHA-512** gehören zur SHA-2-Familie, **SHA-3** ist ein strukturell anderes Verfahren. Diese Verfahren sind für die Integritätsprüfung in der

aktuellen Praxis der Standard. Das NIST beschreibt sie in den Standards FIPS 180-4 und FIPS 202.

> **Gut zu wissen**
>
> Viele forensische Werkzeuge berechnen standardmäßig mehrere Hashwerte gleichzeitig, häufig MD5 und SHA-1 neben SHA-256. Ein Bericht sollte mindestens einen aktuellen Wert enthalten. Ältere Werte können ergänzend mitgeführt werden, damit Vergleiche mit früheren Dokumenten möglich bleiben.

Was ein Hashnachweis belegt und was nicht

Dieser Abschnitt ist der wichtigste des Kapitels. Ein Hashwert beweist einen bestimmten, eng umrissenen Sachverhalt: Die Daten, die zum Zeitpunkt der Berechnung vorlagen, stimmen mit den Daten überein, die zum Zeitpunkt der Berechnung des Vergleichswerts vorlagen. Mehr nicht.

Ein Hashwert beweist insbesondere nicht, dass ein Datenträger vor der Sicherung unverändert war. Wenn das Original bereits verändert war, bevor du es angefasst hast, misst der Hashwert diesen veränderten Zustand. Er beweist auch nicht, wann die Veränderung stattfand, und nicht, wer sie vorgenommen hat. Die Chain of Custody aus Kapitel 3 und das Zustandsprotokoll sind deshalb unverzichtbar.

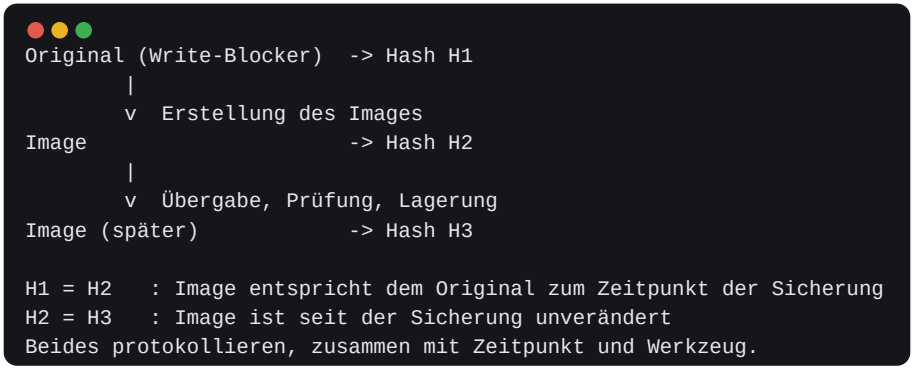
Ein Hashwert des Images belegt außerdem nur, dass das Image seit der Berechnung unverändert ist. Ob das Image vollständig und korrekt aus dem Original entstanden ist, zeigt erst der Vergleich des Original-Hashwerts mit dem des Images, und auch dieser Vergleich setzt voraus, dass der Original-Hashwert vor oder während der Sicherung korrekt berechnet wurde.

Schließlich gilt: Ein gleicher Hashwert ist ein sehr starkes Indiz für Gleichheit. Für SHA-256 ist nach heutigem Stand keine praktische Kollision bekannt, eine absolute Garantie ist das aber nicht. Die Entscheidung, ob ein Nachweis ausreicht, ist eine fachliche und rechtliche Bewertung, die im Praxisprojekt (Kapitel 10) geübt wird.

Hashwerte vor und nach der Sicherung

Der übliche Ablauf besteht aus drei Hashberechnungen. Der erste Wert wird vom Original berechnet, solange es über einen Write-Blocker geschützt ist. Der zweite Wert wird nach der Erstellung des Images vom Image berechnet. Der dritte Wert wird bei jeder späteren Übergabe oder Prüfung wieder vom Image berechnet und mit dem zweiten verglichen. Stimmen der erste und der zweite Wert überein, ist das Image eine vollständige Kopie des Originals zum Zeitpunkt der Sicherung. Stimmt der dritte Wert mit dem zweiten überein, ist das Image seit der Sicherung unverändert.

Diagramm 5.1: Hashbasierte Integritätsprüfung



Hashlisten für Verzeichnisse

Neben dem Hash eines ganzen Images werden häufig Hashwerte einzelner Dateien in einer Liste festgehalten. Eine solche Liste erlaubt es später festzustellen, welche Dateien sich verändert haben, und nicht nur, dass sich etwas verändert hat. Ihre Grenzen sind dieselben wie oben: Sie zeigt Veränderungen seit dem Erstellungszeitpunkt der Liste, nicht seit einem früheren Zeitpunkt.

Zugriffszeiten und Hashberechnung

Auch das Lesen einer Datei kann Spuren verändern. Viele Dateisysteme aktualisieren beim Lesen die Zugriffszeit, je nach Einstellung. Deshalb werden Hashwerte für Beweismittel nach Möglichkeit am Image berechnet, nicht am eingehängten Original. Wo dies nicht möglich ist, muss die Veränderung im Protokoll vermerkt werden.

> **WARNUNG – Sicherheitsrisiko**
>
> Berechne Hashwerte nicht am eingehängten Original, wenn du es vermeiden kannst. Das Einhängen und Lesen kann Zeitstempel und Verzeichniseinträge verändern, bevor du den Hash vergleichst. Arbeite am Image oder an einer Kopie und dokumentiere, ob und wie das Original gelesen wurde.

Typische Fehler

- Ein Hash wird berechnet, aber der Zeitpunkt und das Werkzeug werden nicht festgehalten
- Der Vergleich erfolgt mit einem anderen Verfahren, etwa SHA-1 gegen SHA-256
- Ein MD5-Wert wird als alleiniger Manipulationsnachweis angegeben
- Der Hash des Images wird als Beweis für den Zustand des Originals vor der Sicherung gewertet
- Ein Hash wird nur einmal berechnet und nie wieder geprüft

Praxis: Hashwerte berechnen und prüfen

Die Übungen verwenden das Image `beweis.raw` aus Kapitel 4. Falls du es nicht mehr hast, erzeuge es nach Übung A und B aus Kapitel 4 neu. Arbeite in einem eigenen Verzeichnis.

Übung A: Hashwerte unter Linux berechnen

Lernziel: dieselbe Datei mit mehreren Verfahren hashen und die Unterschiede erkennen. **Voraussetzungen:** Linux-VM mit `beweis.raw`. **Aufgabenstellung:** Berechne den Hashwert des Images mit SHA-256, SHA-512 und zum Vergleich mit MD5.

Befehle zu Übung A

```
$ sha256sum beweis.raw
$ sha512sum beweis.raw
$ md5sum beweis.raw
```

Erklärung: Jedes Werkzeug berechnet einen eigenen Hashwert mit fester Länge. `sha256sum` liefert 64 Hexadezimalzeichen, `sha512sum` 128 und `md5sum` 32. MD5 dient hier nur dem Vergleich; in einem neuen Bericht sollte es nicht als Hauptnachweis erscheinen. **Erwartetes Ergebnis:** Drei Werte unterschiedlicher Länge für dieselbe Datei. **Überprüfung:** Die Längen entsprechen den genannten Werten. Wiederhole den Befehl und vergleiche: Der Wert muss identisch sein. **Fehleranalyse:** Der Wert ändert sich zwischen zwei Läufen? Prüfe, ob die Datei geändert wurde oder ob sie gerade von einem anderen Prozess beschrieben wird.

Übung B: Hashwerte unter Windows mit PowerShell berechnen

Lernziel: denselben Vergleich auf einem Windows-System mit dem eingebauten Befehl durchführen. **Voraussetzungen:** Windows-11-VM mit einer Kopie des Images. **Aufgabenstellung:** Berechne den SHA-256-Wert der Kopie und vergleiche ihn mit dem Wert aus Übung A.

Befehle zu Übung B

```
PS> Get-FileHash -Path .\beweis.raw -Algorithm SHA256
```

Erklärung: `Get-FileHash` ist ein Bordmittel von PowerShell. Der Parameter `-Path` nennt die Datei, `-Algorithm SHA256` wählt das Verfahren ausdrücklich. Setze den Algorithmus immer selbst, damit das Ergebnis nicht von Standardwerten einer Version abhängt. Die Ausgabe enthält Algorithmus, Hashwert und Pfad. **Erwartetes Ergebnis:** Derselbe SHA-256-Wert wie unter Linux, sofern die Datei identisch ist. **Fehleranalyse:** Ein anderer Wert? Prüfe, ob die Kopie vollständig übertragen wurde, etwa über den Vergleich der Dateigröße mit `Get-Item .\beweis.raw`. **Sicherheitsbetrachtung:** Die Kopie muss vorher über ein Medium übertragen werden, dessen Integrität du mit dem Hashwert prüfst. Der Befehl selbst verändert die Datei nicht.

Übung C: Image-Integrität prüfen und eine Veränderung nachweisen

Lernziel: einen Hashnachweis anlegen, prüfen und das Fehlschlagen einer Prüfung erleben. **Voraussetzungen:** `beweis.raw` aus Kapitel 4. **Aufgabenstellung:** Lege einen Hashnachweis für das Image an und prüfe ihn. Danach erzeugst du eine Kopie, veränderst sie absichtlich und prüfst die Kopie gegen den Nachweis des Originals.

Befehle zu Übung C

```
$ sha256sum beweis.raw > beweis.raw.sha256
$ sha256sum -c beweis.raw.sha256
$ cp beweis.raw kopie.raw
$ printf 'X' | dd of=kopie.raw bs=1 seek=1000 count=1 conv=notrunc
$ sed 's/beweis.raw/kopie.raw/' beweis.raw.sha256 >
    kopie.raw.sha256
$ sha256sum -c kopie.raw.sha256
```

Erklärung: Die Hashdatei speichert den Wert und den Dateinamen. `sha256sum -c` berechnet den Wert neu und vergleicht ihn. `dd` mit `seek=1000` schreibt genau ein Byte an Position 1000 der Kopie, `conv=notrunc` verhindert, dass die Datei dabei gekürzt wird. Das Original bleibt unberührt. Die letzte Prüfung nutzt eine Hashdatei, die auf die Kopie zeigt, aber den Wert des Originals enthält. Sie muss daher fehlschlagen. **Erwartetes Ergebnis:** Die erste Prüfung meldet `beweis.raw: OK`. Die letzte Prüfung meldet `kopie.raw: FAILED` und darunter die Warnung, dass eine berechnete Prüfsumme nicht übereinstimmt. **Überprüfung:** `cmp beweis.raw kopie.raw` nennt die Position der Abweichung. Sie muss bei Byte 1001 liegen (Zählung ab 1 bei `cmp`). **Fehleranalyse:** Die letzte Prüfung meldet `OK`? Dann war das Byte an der Position bereits `X`, was selten vorkommt. Wähle eine andere Position oder prüfe mit `cmp`, ob sich die Dateien unterscheiden. **Sicherheitsbetrachtung:** Verändere nie das Original, nur Kopien. Lösche `kopie.raw` und `kopie.raw.sha256` nach der Übung wieder.

Übung D: Hashliste für ein Verzeichnis erstellen und Veränderungen erkennen

Lernziel: nicht nur feststellen, dass etwas verändert wurde, sondern welche Datei betroffen ist. **Voraussetzungen:** ein Verzeichnis mit mindestens drei Dateien, zum Beispiel `testordner` mit eigenen Testdateien. **Aufgabenstellung:** Erstelle eine Hashliste aller Dateien im Verzeichnis, verändere eine Datei und finde sie mit der Prüfung wieder.

Befehle zu Übung D

```
$ find testordner -type f -exec sha256sum {} + > hashliste.sha256
$ sha256sum -c hashliste.sha256
$ echo "Zusatz" >> testordner/datei2.txt
$ sha256sum -c hashliste.sha256
```

Erklärung: `find` sucht alle Dateien (`-type f`) und übergibt sie an `sha256sum`. Die Option `{} +` fügt viele Dateien in einem Aufruf zusammen. Die Prüfung nennt jede Datei einzeln mit `OK` oder `FAILED`. **Erwartetes Ergebnis:** Nach der Veränderung meldet die zweite Prüfung genau `datei2.txt` als nicht übereinstimmend, alle anderen Dateien bleiben `OK`. **Überprüfung:** Die Zahl der geprüften Dateien stimmt mit deiner Zählung überein. **Fehleranalyse:** Dateien mit Leerzeichen im Namen? `find` mit `-print0` und `sha256sum` mit `--null` sind die robustere Variante; für diese Übung genügt die gezeigte Form, weil die Namen keine Leerzeichen enthalten. **Hinweis zur Schreibweise:** Die Hashliste enthält Pfade relativ zum Aufrufverzeichnis. Führe die Prüfung immer aus demselben Verzeichnis aus.

Übung E: Prüfergebnis dokumentieren

Lernziel: ein Prüfergebnis so festhalten, dass ein Dritter es einordnen kann. **Aufgabenstellung:** Schreibe für Übung C ein Protokoll mit folgenden Angaben: Datei, Verfahren, Hashwert, Zeitpunkt mit Zeitzone, verwendete Werkzeuge mit Version (`sha256sum --version`), Ergebnis der Prüfung und eine Aussage, was das Ergebnis belegt und was nicht. **Erwartetes Ergebnis:** Ein Protokoll von höchstens einer halben Seite, in dem die Aussage zu den Grenzen eindeutig formuliert ist. **Fehleranalyse:** Die Aussage lautet „Die Datei ist unverändert“? Ergänze sie um den Zeitpunkt und den Bezugspunkt der Prüfung, sonst ist sie zu weit.

Kontrollfragen

1. Was beweist ein gleicher Hashwert zu zwei Zeitpunkten? (leicht)
2. Warum werden MD5 und SHA-1 für neue Untersuchungen nicht mehr als Hauptnachweis verwendet? (leicht)
3. Warum beweist der Hash des Images nicht allein, dass das Original vor der Sicherung unverändert war? (mittel)
4. Warum berechnet man Hashwerte nach Möglichkeit am Image und nicht am eingehängten Original? (mittel)
5. Eine Hashliste eines Verzeichnisses wurde erst nach einem Vorfall erstellt. Welche Veränderungen kann sie noch aufdecken, und welche nicht? (schwer)

Lösungshinweise: 1. Dass die Daten zu beiden Zeitpunkten übereinstimmen, also dass sich nichts geändert hat, nicht dass sie vorher korrekt waren. 2. Für gezielt herbeigeführte Kollisionen sind beide angreifbar, ein Manipulationsnachweis gegen eine handelnde Person braucht daher ein aktuelles Verfahren. 3. Der Hash misst den Zustand, den das Original zum Zeitpunkt der Sicherung hatte, auch wenn dieser bereits verändert war. 4. Das Einhängen und Lesen kann Zugriffszeiten und Verzeichnisse verändern, bevor der Vergleich stattfindet. 5. Sie zeigt nur Veränderungen seit der Erstellung der Liste, also nichts über Veränderungen davor. Sie kann aber feststellen, welche Dateien heute von dem Stand der Liste abweichen.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] erklären kannst, welchen Sachverhalt ein Hashnachweis belegt und welchen nicht,
- [ ] Hashwerte unter Linux und Windows berechnet und verglichen hast,
- [ ] eine absichtlich veränderte Kopie durch eine fehlgeschlagene Prüfung erkannt hast,
- [ ] ein Prüfergebnis so dokumentierst, dass Zeitpunkt, Werkzeug und Aussagegrenze eindeutig sind.

Zusammenfassung

- Ein Hashwert belegt Übereinstimmung zu einem dokumentierten Zeitpunkt, nicht die Unversehrtheit vor der Sicherung.
- Für neue Untersuchungen sind SHA-256 oder neuere Verfahren der Standard, MD5 und SHA-1 dienen höchstens dem Vergleich mit älteren Dokumenten.
- Hashwerte entstehen am besten am Image, damit das Original nicht durch Lesezugriffe verändert wird.
- Hashlisten zeigen, welche Dateien sich verändert haben, und ergänzen den Nachweis für ganze Datenträger.

Kapitel 6: Windows-Forensik

Lernziele

- Du erklärst, welche Spuren Windows typischerweise hinterlässt und wie zuverlässig sie sind.
- Du liest Ereignisprotokolle, Registry-Einträge, Prefetch-Dateien und die USB-Gerätehistorie aus und ordnest sie zeitlich ein.
- Du benennst für jedes Artefakt, was es belegt und was nicht.

WARNUNG – Sicherheitsrisiko

Die Übungen dieses Kapitels lesen nur Daten. Führe sie ausschließlich auf deinem Windows-Testsystem aus. Auf einem beschlagnahmten oder produktiven System gelten die Regeln aus Kapitel 3 und 5: Arbeite möglichst am Image und dokumentiere jeden Zugriff, auch wenn er nur lesend ist.

Die Spuren eines Windows-Systems

Windows protokolliert viel, aber nicht alles, und nicht immer in der Form, die man sich wünscht. Die wichtigsten Spurenquellen lassen sich in drei Gruppen einteilen. Ereignisprotokolle zeichnen Vorgänge wie Anmeldungen und Systemereignisse auf. Die Registry speichert Konfiguration und Verläufe, etwa welche Geräte angeschlossen wurden. Dateien wie Prefetch-Einträge halten fest, welche Programme ausgeführt wurden. Jede Gruppe hat eigene Stärken und Lücken.

Diagramm 6.1: Windows-Artefakte im Überblick (Auswahl)

```
Ereignisprotokolle
  Anmeldungen, Dienste, Systemereignisse
  C:\Windows\System32\winevt\Logs
Registry
  Geräte, Benutzerkonfiguration, Verläufe
  C:\Windows\System32\config, NTUSER.DAT
Prefetch
  Programmstarts
  C:\Windows\Prefetch
Gerätehistorie
  erstmalige Installation von USB-Geräten
  C:\Windows\INF\setupapi.dev.log
```

Diese Pfade gelten für Windows 10 und Windows 11 in der Standardinstallation. Prüfe bei einem fremden System immer, ob die Pfade mit der tatsächlichen Konfiguration übereinstimmen.

Ereignisprotokolle

Die Ereignisprotokolle liegen als Dateien mit der Endung `.evtx` im Verzeichnis `C:\Windows\System32\winevt\Logs`. Für die Untersuchung sind drei besonders wichtig: `Security.evtx` (Anmeldungen, Rechteänderungen, Richtlinienereignisse), `System.evtx` (Dienste, Treiber, Systemstart und -herunterfahren) und `Application.evtx` (Anwendungsereignisse).

Jedes Ereignis trägt eine Ereignis-ID. Einige IDs sind für die Untersuchung von Anmeldungen zentral:

- **4624**: erfolgreiche Anmeldung
- **4625**: fehlgeschlagene Anmeldung
- **4634** und **4647**: Abmeldung
- **4648**: Anmeldung mit explizit angegebenen Zugangsdaten
- **4672**: Zuweisung besonderer Rechte bei der Anmeldung
- **1102** im Security-Protokoll: das Sicherheitsprotokoll wurde gelöscht

Die Logontypen im Feld des Ereignisses 4624 verraten, wie die Anmeldung erfolgte. Typ 2 steht für eine interaktive Anmeldung am Gerät, Typ 3 für eine Netzwerkanmeldung, Typ 7 für das Entsperren einer Sitzung und Typ 10 für

eine Remotedesktop-Anmeldung. Typ 10 ist deshalb für die Frage nach Fernzugriffen besonders aussagekräftig.

> **Gut zu wissen**
>
> Ob ein Vorgang wie die Prozesserstellung (Ereignis 4688) protokolliert wird, hängt von den Überwachungsrichtlinien ab, die im System gesetzt sind. Fehlt ein erwartetes Ereignis, prüfe zuerst die Richtlinie, bevor du daraus schließt, dass der Vorgang nicht stattfand.

Die Zeitstempel in Ereignisprotokollen sind in UTC gespeichert. Die Anzeige in der Ereignisanzeige erfolgt dagegen in lokaler Zeit. Das führt bei Auswertungen regelmäßig zu Verwechslungen, wie Kapitel 8 zeigt. Notiere bei jeder Auswertung, in welcher Zeitzone du die Werte siehst.

Was Ereignisprotokolle belegen und was nicht

Ein Ereignis 4624 mit Logontyp 10 belegt, dass zu einem bestimmten Zeitpunkt eine Remotedesktop-Anmeldung für das angegebene Konto erfolgreich war. Es belegt nicht, wer am anderen Ende saß, und es belegt nicht, was danach getan wurde. Ein Konto kann auch ohne Wissen seines Inhabers verwendet worden sein. Die Aussage „Konto X hat sich angemeldet“ ist also eine Beobachtung, „Person X hat sich angemeldet“ wäre eine Schlussfolgerung, die weitere Belege braucht.

Das Löschen eines Protokolls mit Ereignis 1102 ist selbst ein Ereignis, das sich auswerten lässt. Fehlt dieses Ereignis, obwohl Lücken im Protokoll auffallen, ist das ein Hinweis, der geprüft werden muss, aber kein Beweis.

Registry

Die Registry ist eine hierarchische Datenbank für Konfiguration und Zustand des Systems. Sie besteht aus Dateien, den sogenannten Hives. Die wichtigsten sind `SYSTEM`, `SOFTWARE`, `SAM` und `SECURITY` im Verzeichnis `C:\Windows\System32\config`, dazu die benutzerspezifische Datei `NTUSER.DAT` im Profilverzeichnis jedes Benutzers.

Für Registry-Einträge gibt es einen Zeitstempel, die sogenannte letzte Schreibzeit des Schlüssels. Er zeigt, wann der Schlüssel zuletzt geändert wurde. Er ist kein Zeitstempel für jeden einzelnen Wert im Schlüssel, und er wird durch andere Änderungen am selben Schlüssel überschrieben. Das ist

eine wichtige Einschränkung, die bei jeder Auswertung im Kopf bleiben muss.

> **WARNUNG – Sicherheitsrisiko**
>
> Lade ein Hive nicht in die Registry des Untersuchungsrechners, etwa mit `reg load`, wenn es sich um Beweismaterial handelt. Das Laden kann die Datei verändern, etwa wenn ausstehende Transaktionsprotokolle eingespielt werden. Lies Hives aus einer Kopie oder mit einem Werkzeug, das den Inhalt ohne Laden auswertet, und dokumentiere das Vorgehen.

Was die Registry belegen und was nicht

Ein Registry-Eintrag belegt, dass eine Einstellung oder ein Gerät zu einem bestimmten Zeitpunkt in der Registry vorhanden war. Er belegt nicht, dass der Eintrag durch einen bestimmten Benutzer erzeugt wurde, außer der Schlüssel liegt im Profil dieses Benutzers. Er belegt auch nicht, dass eine Funktion tatsächlich genutzt wurde, nur weil sie konfiguriert ist.

Prefetch

Prefetch ist ein Mechanismus, mit dem Windows den Programmstart beschleunigt, indem es merkt, welche Dateien ein Programm beim Start braucht. Dazu legt das System Dateien mit der Endung `.pf` im Verzeichnis `C:\Windows\Prefetch` an. Ein Dateiname hat die Form `PROGRAMMNAME-HASH.pf`, wobei der Hash aus dem Pfad des Programms gebildet wird, nicht aus seinem Inhalt.

Eine Prefetch-Datei enthält typischerweise die Anzahl der Starts, die Zeitpunkte der letzten Starts (bei neueren Windows-Versionen mehrere) und die Dateien und Verzeichnisse, auf die das Programm zugegriffen hat. Diese Angaben machen Prefetch für die Untersuchung nützlich, besonders bei Programmen, die sich nicht sauber protokollieren.

> **Gut zu wissen**
>
> Prefetch ist eine Systemfunktion für die Startoptimierung, keine Protokollierung für forensische Zwecke. Ob sie aktiv ist, hängt von Version, Installation und Konfiguration ab. Ein fehlender Prefetch-Eintrag beweist deshalb nicht, dass ein Programm nicht gestartet wurde. Werkzeuge wie PECmd aus dem Umfeld von Eric Zimmerman lesen Prefetch-Dateien aus. Prüfe vor dem Einsatz Herkunft und Version des Werkzeugs.

Was Prefetch belegen und was nicht

Ein Prefetch-Eintrag zeigt, dass ein Programm auf diesem System gestartet wurde, und zu welchen Zeiten. Er zeigt nicht, wer es gestartet hat, ob es erfolgreich lief, oder welche Daten es verarbeitet hat. Die Liste der zugegriffenen Dateien ist ein Hinweis auf Arbeitsabläufe, aber kein vollständiger Nachweis dafür.

USB-Gerätehistorie

Wenn ein USB-Gerät zum ersten Mal an einem Windows-System angeschlossen wird, installiert Windows den passenden Treiber. Dieser Vorgang wird in der Datei `C:\Windows\INF\setupapi.dev.log` mit Zeitangaben festgehalten. Die Datei ist ein Textprotokoll und lässt sich mit Bordmitteln durchsuchen. Die Gerätekennung steht in der Form `USB\VID_xxxx&PID;_xxxx` für Vendor- und Produktkennung.

Zusätzlich speichert die Registry Informationen über USB-Massenspeicher im Schlüssel `HKLM\SYSTEM\CurrentControlSet\Enum\USBSTOR`. Dort stehen Gerätetyp, Hersteller, Produkt, Version und die Seriennummer des Geräts. Der Eintrag bleibt auch nach dem Entfernen des Sticks bestehen, bis er gelöscht wird.

Der Eintrag belegt damit, dass ein Gerät mit dieser Seriennummer an diesem System angeschlossen war. Er belegt nicht, welche Daten übertragen wurden, ob Dateien kopiert, geöffnet oder nur gelesen wurden und wer das Gerät angeschlossen hat. Dafür braucht man weitere Spuren, zum Beispiel Ereignisse der Gerätenutzung, Dateizugriffe im Benutzerprofil oder Verknüpfungen zu geöffneten Dateien.

> **WARNUNG – Sicherheitsrisiko**
>
> Die Seriennummer aus dem Registry-Eintrag ist nicht zwingend eindeutig und kann bei manchen Geräten vom System erzeugt sein. Vergleiche sie deshalb immer mit dem tatsächlichen Gerät, und prüfe auch den Gerätetyp und die Kapazität.

Weitere Artefakte, nur mit Grenzen

Windows enthält weitere Spuren, die in dieser Einführung nicht vertieft werden. Dazu gehören Amcache und Shimcache, die Programme und Kompatibilitätsdaten speichern, LNK-Dateien und Jump Lists, die Zugriffe auf Dateien über Verknüpfungen festhalten, und der Browserverlauf. Ihre Auswertung ist anspruchsvoller, und ihre Bedeutung hängt stark von Version und Konfiguration ab. Wer sie auswertet, sollte die Dokumentation des jeweiligen Werkzeugs und die Einschränkungen der Windows-Version genau kennen.

Praxis: Windows-Spuren auswerten

Die Übungen laufen auf einer Windows-11-VM. Erstelle vor den Übungen einen Snapshot der VM, damit du den Ausgangszustand jederzeit wiederherstellen kannst. Das ist selbst eine Form der Reproduzierbarkeit. Verwende für Übung D einen eigenen USB-Stick.

Übung A: Anmeldeereignisse aus dem Sicherheitsprotokoll lesen

Lernziel: Anmeldeereignisse mit Logontyp und Zeitangabe auslesen. **Voraussetzungen:** Windows-11-VM, administrative Rechte, ein paar Anmeldungen in der VM (zum Beispiel eine Abmeldung und erneute Anmeldung). **Aufgabenstellung:** Lies die letzten zehn erfolgreichen Anmeldeereignisse aus dem Sicherheitsprotokoll.

Befehle zu Übung A

```
PS> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624}
    -MaxEvents 10 |
    Select-Object TimeCreated, Message
```

Erklärung: `-FilterHashtable` filtert nach Protokoll und Ereignis-ID. `-MaxEvents 10` begrenzt die Ausgabe. Das Feld `TimeCreated` wird in lokaler Zeit angezeigt. Im Feld `Message` steht der Logontyp, den du im Abschnitt Ereignisprotokolle nachschlagen kannst. **Erwartetes Ergebnis:** Eine Liste mit Zeitpunkt und Beschreibung, in der die Anmeldung aus deiner Übung als Typ 2 auftaucht. **Überprüfung:** Die Zeit stimmt mit dem Zeitpunkt deiner Anmeldung überein, unter Berücksichtigung der Zeitzonenanzeige. **Fehleranalyse:** Keine Ereignisse? Prüfe, ob die Ereignisprotokollierung für Anmeldungen aktiv ist (Überwachungsrichtlinie). Auch zu geringe Rechte können die Ausgabe leer lassen: starte PowerShell als Administrator.

Übung B: Ereignisprotokolle aus einer Kopie offline auswerten

Lernziel: eine .evtx-Datei lesen, ohne das Protokoll am laufenden System zu nutzen. **Voraussetzungen:** eine Kopie der Datei `Security.evtx` (bei dieser Übung kopierst du sie aus der VM auf den Untersuchungsrechner und prüfst den Hash vorher). **Aufgabenstellung:** Lies die letzten 20 Ereignisse aus der kopierten Datei aus.

Befehle zu Übung B

```
PS> Get-FileHash -Path .\Security.evtx -Algorithm SHA256
PS> Get-WinEvent -Path .\Security.evtx -MaxEvents 20 |
    Select-Object TimeCreated, Id, Message
```

Erklärung: Der erste Befehl dokumentiert den Hashwert der Kopie, wie in Kapitel 5 gezeigt. Der zweite liest die Kopie mit `-Path`, was einen Dateipfad statt eines laufenden Protokolls verwendet. Auf diese Weise bleibt das Original unverändert, und die Auswertung lässt sich mit dem Hashwert belegen. **Erwartetes Ergebnis:** Ein Hashwert und eine Liste der letzten Ereignisse mit Ereignis-ID. **Überprüfung:** Die Ereignisse aus Übung A erscheinen in der Liste. **Fehleranalyse:** Die Datei ist gesperrt und lässt sich nicht kopieren? Das laufende System hält sie offen. Verwende dann eine Kopie aus einem Snapshot oder einem Image, wie in Kapitel 4 beschrieben.

Übung C: Die Registry nach USB-Gerätespuren durchsuchen

Lernziel: den USBSTOR-Schlüssel lesen und die Angaben eines Geräts einordnen. **Voraussetzungen:** Windows-11-VM, ein USB-Stick, der zuvor an der VM angeschlossen war. **Aufgabenstellung:** Lies den Schlüssel USBSTOR aus und suche den Eintrag deines Teststicks.

Befehle zu Übung C

```
PS> reg query "HKLM\SYSTEM\CurrentControlSet\Enum\USBSTOR" /s
```

Erklärung: `reg query` liest Schlüssel und Werte, `/s` durchsucht auch alle Unterschlüssel. Der Befehl verändert nichts. Jeder Gerätetyp erscheint als eigener Unterschlüssel, die Seriennummer im Namen des Unterschlüssels der zweiten Ebene. **Erwartetes Ergebnis:** Ein Eintrag mit dem Herstellernamen und Produktnamen deines Sticks, die Seriennummer im Schlüsselnamen. **Überprüfung:** Vergleiche die Seriennummer mit der Angabe auf dem Stick oder mit der Ausgabe von `Get-Disk` beziehungsweise dem Eintrag in der Datenträgerverwaltung. **Fehleranalyse:** Der Eintrag fehlt? Prüfe, ob der Stick tatsächlich über den Massenspeicher-Treiber erkannt wurde, und ob du das richtige Laufwerk ausgewählt hast. **Sicherheitsbetrachtung:** Lösche in dieser Übung keine Schlüssel. Auf einem beschlagnahmten System ist jede Änderung an der Registry eine Veränderung von Beweismaterial.

Übung D: Prefetch-Dateien auflisten und einordnen

Lernziel: Programmstarts aus Prefetch-Dateien ablesen und mit den Anmeldungen aus Übung A vergleichen. **Voraussetzungen:** Windows-11-VM mit administrativen Rechten, mindestens ein Programm, das du vor der Übung gestartet hast (zum Beispiel den Rechner-Editor). **Aufgabenstellung:** Liste die Prefetch-Dateien mit Namen und Änderungszeit auf.

Befehle zu Übung D

```
PS> Get-ChildItem C:\Windows\Prefetch -Filter *.pf |
    Select-Object Name, LastWriteTime | Sort-Object LastWriteTime
        -Descending | Select-Object -First 10
```

Erklärung: `Get-ChildItem` listet die Dateien, `-Filter *.pf` begrenzt auf Prefetch-Dateien. `LastWriteTime` ist der Zeitstempel der Datei, nicht der letzte Programmstart im Inhalt. Für die Starts selbst braucht es ein Werkzeug, das die Datei liest (etwa PECmd). Die Namen zeigen die Programmnamen mit dem Hash des Pfads. **Erwartetes Ergebnis:** Eine Liste mit Programmnamen, die zu den Programmen passen, die du gestartet hast. **Überprüfung:** Stimmt der Zeitstempel ungefähr mit deinem Start überein? Beachte, dass ältere Dateien eventuell nicht mehr vorhanden sind, weil Windows sie verwaltet. **Fehleranalyse:** Keine Dateien? Prüfe, ob Prefetch in der VM aktiviert ist. Das ist in der Standardkonfiguration von Windows 11 üblich, aber nicht garantiert. **Hinweis:** Wenn du Starts mit Uhrzeiten auswerten willst, brauchst du ein Prefetch-Lesewerkzeug. Die Zeitangabe `LastWriteTime` allein reicht dafür nicht.

Typische Fehler

- Eine Anmeldung wird als Handlung einer bestimmten Person dargestellt, obwohl sie nur einem Konto zugeordnet ist.
- Zeitstempel aus Ereignisprotokollen werden ohne Umrechnung von UTC in lokale Zeit übernommen.
- Ein fehlender Eintrag wird als Beweis dafür gewertet, dass etwas nicht stattfand.
- Eine Registry-Datei wird geladen, wodurch sie verändert wird.
- Die Seriennummer eines USB-Geräts wird nicht mit dem physischen Gerät abgeglichen.

> **Zusammenfassung**
>
> - Ereignisprotokolle zeigen Anmeldungen und Systemereignisse. Der Logontyp verrät, wie die Anmeldung erfolgte, die Zeitstempel sind in UTC gespeichert.
> - Registry-Zeitstempel beziehen sich auf den Schlüssel, nicht auf einzelne Werte.
> - Prefetch belegt Programmstarts, nicht deren Urheber oder Ergebnis, und hängt von der Konfiguration ab.
> - Die USB-Gerätehistorie belegt, dass ein Gerät angeschlossen war, nicht, welche Daten übertragen wurden.

Kontrollfragen

1. Welche Datei enthält das Sicherheitsprotokoll, und welche Ereignis-ID markiert eine erfolgreiche Anmeldung? (leicht)
2. Was bedeutet Logontyp 10 in einem Anmeldeereignis? (leicht)
3. Warum ist die letzte Schreibzeit eines Registry-Schlüssels kein Zeitstempel für einzelne Werte? (mittel)
4. Ein Prefetch-Eintrag zeigt einen Programmstart um 14:05 UTC. Was belegt er, und was nicht? (mittel)
5. Ein USBSTOR-Eintrag ist vorhanden, aber das Gerät wird nicht mehr gefunden. Welche Schlüsse sind zulässig, welche nicht? (schwer)

Lösungshinweise: 1. `Security.evtx`, Ereignis-ID 4624. 2. Eine Remotedesktop-Anmeldung. 3. Die Zeit bezieht sich auf die letzte Änderung irgendeines Werts im Schlüssel, weshalb sie einzelne Werte nicht zuverlässig datiert. 4. Dass das Programm um diese Zeit gestartet wurde. Nicht belegt sind Benutzer, Erfolg und verarbeitete Daten. 5. Zulässig ist: Das Gerät war zu einem früheren Zeitpunkt an diesem System angeschlossen. Nicht zulässig ist: Das Gerät existiert nicht mehr oder wurde entfernt, oder es wurden Daten übertragen. Der Eintrag bleibt auch nach dem Entfernen bestehen.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Anmeldeereignisse mit Ereignis-ID und Logontyp auslesen und zeitlich einordnen kannst,
- [ ] eine Protokolldatei aus einer Kopie auswerten kannst, ohne das Original zu verändern,
- [ ] den USBSTOR-Eintrag eines Geräts finden und seine Aussagegrenze benennen kannst,
- [ ] für jedes der vier Artefakte angeben kannst, was es belegt und was nicht.

Kapitel 7: Linux-Forensik

Lernziele

- Du erklärst, welche Spuren ein Linux-System typischerweise hinterlässt und wo sie liegen.
- Du wertest Systemprotokolle, Authentifizierungsereignisse, Dateimetadaten und Paketinstallationen aus.
- Du benennst für jede Spur, was sie belegt, und erkennst, wann eine Spur fehlt, ohne dass das etwas beweist.

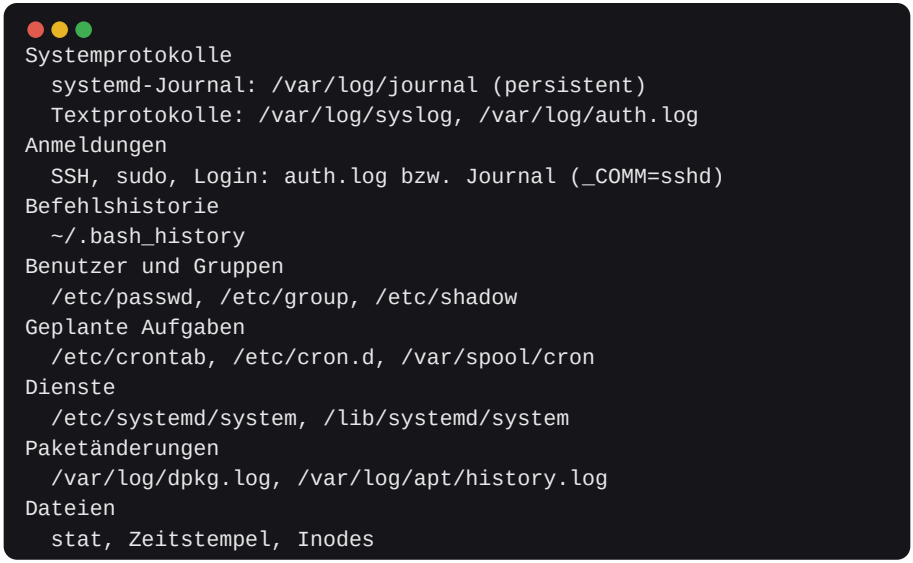
WARNUNG – Sicherheitsrisiko

Die Übungen lesen nur Daten. Führe sie auf deinem Linux-Testsystem aus, und arbeite bei Beweismaterial am Image, wie in Kapitel 4 beschrieben. Das Lesen von Dateien kann Zugriffszeiten verändern, deshalb gilt auch hier Kapitel 5: Dokumentiere jeden Zugriff.

Einordnung

Linux ist kein einheitliches System. Die Spuren hängen von der Distribution, vom Init-System und von der Protokollierung ab. Dieses Kapitel beschreibt Debian und Ubuntu, weil sie auf Servern und in Schulungsumgebungen besonders häufig sind. Für andere Familien, etwa Fedora oder Red Hat Enterprise Linux, gibt es vergleichbare Spuren, aber teils andere Dateinamen. Prüfe bei jedem fremden System zuerst, welche Distribution und welche Version vorliegen, etwa über `/etc/os-release`, und welche Protokollierung aktiv ist.

Diagramm 7.1: Linux-Spuren im Überblick (Debian und Ubuntu)



Systemprotokolle und das Journal

Viele moderne Linux-Systeme schreiben ihre Protokolle in das systemd-Journal. Es speichert Ereignisse in einem binären Format und ist mit `journalctl` auszulesen. Ob das Journal dauerhaft gespeichert wird, hängt von der Konfiguration ab. Liegt es unter `/var/log/journal`, überlebt es einen Neustart. Liegt es unter `/run/log/journal`, geht es beim Neustart verloren. Diese Unterscheidung ist für die Sicherung entscheidend.

Viele Systeme schreiben zusätzlich Textprotokolle über rsyslog, etwa `/var/log/syslog`. Ob diese Dateien existieren, hängt davon ab, ob rsyslog installiert ist. Auf reinen Journal-Systemen fehlen sie. Prüfe also, welche Quelle tatsächlich vorhanden ist, bevor du sie auswertest.

Ein Journal aus einer Kopie liest du mit `journalctl --directory` (kurz `-D`) und dem Pfad zum Verzeichnis, das die Journaldateien enthält. Die Zeitanzeige folgt der lokalen Zeitzone. Mit `--utc` erhältst du UTC, was für den Vergleich mit anderen Quellen meist sinnvoller ist.

Anmeldungen und Authentifizierung

Authentifizierungsereignisse stehen bei Debian und Ubuntu typischerweise in `/var/log/auth.log`, sofern rsyslog aktiv ist. Ohne rsyslog finden sie sich im Journal, zum Beispiel über das Feld des Prozessnamens `_COMM=sshd`. Typische Einträge für SSH sehen so aus:

- `Accepted publickey for ...`: erfolgreiche Anmeldung mit Schlüssel
- `Accepted password for ...`: erfolgreiche Anmeldung mit Passwort
- `Failed password for ...`: fehlgeschlagener Versuch mit Passwort
- `sudo: ... COMMAND=...`: Ausführung eines Befehls mit erhöhten Rechten

Die genauen Formulierungen können sich zwischen Versionen ändern. Vergleiche Auszüge immer mit einer Vergleichsanmeldung im eigenen Testsystem, bevor du daraus Regeln ableitest.

Was Anmeldeereignisse belegen und was nicht

Ein `Accepted publickey`-Eintrag belegt, dass sich für ein Konto ein Schlüssel erfolgreich authentifiziert hat. Welcher Schlüssel das war, steht im Eintrag nur als Fingerabdruck. Wer den Schlüssel besaß, geht daraus nicht hervor. Dafür braucht es die `authorized_keys`-Datei des Kontos und eine Zuordnung zu einer Person, die gesondert belegt werden muss.

Fehlende Einträge sind kein Beweis für das Fehlen von Anmeldungen. Protokolle rotieren, werden gekürzt oder fehlen ganz, wenn ein Dienst nicht protokolliert hat.

Befehlshistorie

Die Shell speichert ihre Befehle in `~/.bash_history` für jeden Benutzer. Das geschieht in der Regel beim Beenden der Shell, nicht fortlaufend. Ein Befehl, der in einer Sitzung ausgeführt wurde, die noch läuft oder nicht sauber beendet wurde, fehlt deshalb in der Datei. Zeitangaben stehen nur darin, wenn die Variable `HISTTIMEFORMAT` gesetzt war. Außerdem lässt sich die Historie durch Einstellungen deaktivieren oder durch Löschen der Datei verändern.

Die Befehlshistorie ist also eine Hinweisquelle, kein vollständiges Protokoll. Sie belegt, dass ein Befehl in einer Sitzung eingegeben wurde, sofern er dort gespeichert ist. Ob er erfolgreich war oder ob er tatsächlich ausgeführt wurde, zeigt sie nicht.

Benutzer, Gruppen und Anmeldedaten

Die Datei `/etc/passwd` enthält pro Zeile ein Konto mit Benutzername, Benutzer-ID, Gruppen-ID, Kommentarfeld, Heimatverzeichnis und Standard-Shell. Konten mit einer Shell wie `/bin/bash` sind interaktiv nutzbar, Konten mit `/usr/sbin/nologin` nicht. Die Datei `/etc/group` enthält Gruppen und ihre Mitglieder.

Die Datei `/etc/shadow` enthält Passwort-Hashes und ist nur für root lesbar. Sie ist in einer Untersuchung sensibel: Sie enthält Angaben, die über den Auftrag hinaus nicht gebraucht werden, und sie darf nicht unbegrenzt verbreitet werden. Lies sie nur, wenn der Auftrag es verlangt, und dokumentiere das. Hashes daraus zu knacken ist kein Teil dieser Untersuchung.

Geplante Aufgaben

Mit cron laufen Aufgaben regelmäßig. Systemweite Aufgaben stehen in `/etc/crontab` und in den Dateien unter `/etc/cron.d`. Benutzerspezifische Aufgaben liegen unter `/var/spool/cron/crontabs/`, dort mit dem Kontonamen als Dateinamen. Eine unbekannte Aufgabe in einer dieser Dateien ist ein Befund, der geprüft werden muss. Eine Aufgabe allein beweist aber nicht, dass sie ausgeführt wurde. Dafür braucht es Protokolle des cron-Dienstes oder Einträge im Journal.

Dienste und Autostart

Unter systemd sind Dienste als Unit-Dateien beschrieben. Aktivierte Dienste erkennst du an Verknüpfungen, die im Verzeichnis `/etc/systemd/system` liegen. Die Standard-Unit-Dateien liegen unter `/lib/systemd/system`. Ein fremder oder ungewöhnlich benannter Dienst in `/etc/systemd/system` verdient eine genauere Prüfung, besonders wenn er auf eine Datei außerhalb der üblichen Pfade verweist.

Paketinstallationen

Auf Debian und Ubuntu protokolliert `dpkg` jede Paketänderung in `/var/log/dpkg.log`, die Datei `/var/log/apt/history.log` ergänzt die Aktionen von apt. Ältere Protokolle sind häufig komprimiert, mit `zcat` oder `zgrep` lassen sie sich trotzdem lesen. Paketinstallationen sind wichtig, weil ein manuell installiertes Werkzeug auf eine Absicht hindeuten kann. Sie sind aber auch alltäglich, und ein einzelnes Paket belegt noch keine Absicht.

Dateimetadaten

Jede Datei hat Metadaten im Inode: Größe, Eigentümer, Rechte und drei Zeitstempel. Der Befehl `stat` zeigt sie an. Die drei Zeiten bedeuten:

- **Access** (Zugriff): wann die Datei zuletzt gelesen wurde
- **Modify** (Änderung des Inhalts): wann der Inhalt zuletzt geschrieben wurde
- **Change** (Änderung des Inodes): wann Metadaten wie Rechte oder Eigentümer zuletzt geändert wurden

Der Change-Zeitstempel wird oft mit einer Erstellungszeit verwechselt, ist aber keine. Er wird auch bei einer Rechteänderung gesetzt, nicht nur beim Anlegen. Eine Erstellungszeit (Birth) zeigt `stat` nur, wenn das Dateisystem sie speichert. Ob ext4 sie liefert, hängt von Version und Werkzeug ab. Prüfe das in deinem Testsystem, bevor du dich darauf verlässt.

Die Inodenummer identifiziert die Datei auf dem Dateisystem. Sie ist nützlich, wenn du eine Datei unter einem anderen Namen wiederfinden willst, etwa nach einer Umbenennung.

> **Gut zu wissen**
>
> Metadaten lassen sich durch Befehle wie `touch` verändern, die Zeitstempel setzen können. Ein Change-Zeitstempel, der zu einem auffälligen Zeitpunkt passt, ist deshalb ein Hinweis, der mit anderen Quellen abgeglichen werden muss. Die Zeitstempel allein sind kein Beweis.

Zeitgleiche Ereignisse im Journal

Ereignisse aus dem Journal, aus Textprotokollen und aus den Metadaten sind die Grundlage für eine Timeline. Kapitel 8 führt sie zusammen. Wichtig ist hier schon: Die Zeitangaben aus verschiedenen Quellen müssen auf dieselbe Zeitbasis umgerechnet werden, bevor sie verglichen werden.

Typische Fehler

- Die Protokollquelle wird nicht geprüft, und ein Textprotokoll fehlt, weil das System nur das Journal nutzt.
- Die Befehlshistorie wird als vollständige Liste aller Befehle dargestellt.
- Der Change-Zeitstempel wird als Erstellungszeit ausgegeben.
- `/etc/shadow` wird ohne Auftrag gelesen oder kopiert.
- Eine Distributionsangabe wird übernommen, ohne sie mit `/etc/os-release` zu prüfen.

> **Zusammenfassung**
>
> - Das systemd-Journal ist die wichtigste Protokollquelle auf modernen Systemen. Ob es persistent ist, entscheidet, ob es einen Neustart überlebt.
> - Authentifizierungsereignisse stehen in auth.log oder im Journal. Sie belegen Anmeldungen einem Konto zu, nicht einer Person.
> - Die Befehlshistorie wird beim Beenden der Shell geschrieben und ist keine vollständige Aufzeichnung.
> - Der Change-Zeitstempel ist keine Erstellungszeit. Metadaten sind leicht veränderbar und brauchen Abgleich mit anderen Quellen.

Praxis: Linux-Spuren auswerten

Die Übungen laufen auf einem Linux-Testsystem, zum Beispiel einer Debian-VM. Lege vorher einen Snapshot an. Für Anmeldeereignisse brauchst du einen SSH-Dienst, der läuft, und einen Testbenutzer. Wenn du den Dienst nicht einrichten willst, kannst du die Übungen für die lokalen Befehle trotzdem durchführen.

Übung A: Systemereignisse aus dem Journal lesen

Lernziel: einen Zeitraum aus dem Journal auswerten und dabei die Zeitzone kontrollieren. **Voraussetzungen:** Linux-Testsystem mit systemd. **Aufgabenstellung:** Lies die Journalereignisse der letzten Stunde in UTC aus.

Befehle zu Übung A

```
$ journalctl --utc --since "1 hour ago" --no-pager | head -n 40
```

Erklärung: `--utc` zeigt die Zeitstempel in UTC, `--since` begrenzt den Zeitraum, `--no-pager` verhindert die interaktive Ansicht. `head` begrenzt die Ausgabe für die Übung. **Erwartetes Ergebnis:** Zeilen mit Zeitstempel, Rechnername, Prozessname und Meldung. **Überprüfung:** Vergleiche die Angabe mit `date -u`. Die Zeiten müssen zueinander passen. **Fehleranalyse:** Keine Ausgabe? Prüfe mit `journalctl --disk-usage`, ob ein Journal vorhanden ist. Ohne Journal fehlt die Quelle, das ist selbst ein Befund.

Übung B: Anmeldeereignisse von SSH herausfiltern

Lernziel: erfolgreiche und fehlgeschlagene SSH-Anmeldungen aus dem Journal filtern. **Voraussetzungen:** SSH-Dienst läuft, mindestens eine Anmeldung mit Passwort oder Schlüssel und einen fehlgeschlagenen Versuch. **Aufgabenstellung:** Filtere die Einträge des SSH-Prozesses und suche nach den Zeilen mit Anmeldungen.

Befehle zu Übung B

```
$ journalctl --utc _COMM=sshd --no-pager | grep -E
    "Accepted|Failed"
```

Erklärung: `_COMM=sshd` filtert nach dem Prozessnamen, das ist ein Feld des Journals. `grep -E` sucht nach beiden Mustern. Auf einem System mit auth.log kannst du stattdessen `grep -E "Accepted|Failed" /var/log/auth.log` verwenden. **Erwartetes Ergebnis:** Eine Zeile je Anmeldeversuch mit Zeitstempel, Konto und Herkunftsadresse. **Überprüfung:** Die Anmeldung aus deiner Übung erscheint mit dem richtigen Konto und dem Zeitpunkt. **Fehleranalyse:** Keine Zeilen? Prüfe mit `systemctl status ssh` (Debian) beziehungsweise `systemctl status`

`sshd` (andere Distributionen), ob der Dienst läuft, und ob der Prozessname im Journal tatsächlich `sshd` lautet.

Übung C: Dateimetadaten auslesen

Lernziel: die drei Zeitstempel einer Datei lesen und ihre Bedeutung unterscheiden. **Aufgabenstellung:** Lies die Metadaten von `/etc/passwd` aus und ändere danach die Rechte einer eigenen Testdatei, um den Change-Zeitstempel zu beobachten.

Befehle zu Übung C

```
$ stat /etc/passwd
$ echo "Test" > testdatei.txt
$ stat testdatei.txt
$ chmod 600 testdatei.txt
$ stat testdatei.txt
```

Erklärung: Die Ausgabe von `stat` enthält die Zeitstempel Access, Modify und Change. Das Lesen von `/etc/passwd` kann den Access-Zeitstempel ändern, je nach Mount-Option des Dateisystems, nicht aber den Inhalt. Das Ändern der Rechte setzt den Change-Zeitstempel neu, der Modify-Zeitstempel bleibt gleich. **Erwartetes Ergebnis:** Nach `chmod` ist der Change-Zeitstempel neuer als vorher, der Modify-Zeitstempel unverändert. **Überprüfung:** Vergleiche beide Ausgaben von `stat testdatei.txt`. **Fehleranalyse:** Der Modify-Zeitstempel hat sich verändert? Prüfe, ob du die Datei erneut beschrieben hast. **Sicherheitsbetrachtung:** Verändere in dieser Übung keine Systemdateien, nur die Testdatei.

Übung D: Dateien mit neuerem Zeitstempel finden

Lernziel: Änderungen in einem Verzeichnis über Zeitstempel finden. **Aufgabenstellung:** Suche in `/etc` nach Dateien, deren Inhalt in den letzten sieben Tagen geändert wurde.

Befehle zu Übung D

```
$ sudo find /etc -type f -mtime -7 -ls
```

Erklärung: `-mtime -7` wählt Dateien, deren Inhalt (Modify-Zeitstempel) in den letzten sieben Tagen geändert wurde. `-ls` gibt die Metadaten in einer

langen Liste aus. Für die Änderung der Metadaten gibt es `-ctime`, für Zugriffe `-atime`. **Erwartetes Ergebnis:** Eine Liste der Dateien, die in dieser Zeit geändert wurden. Auf einem frisch installierten System können das einige Dateien sein, die während der Installation entstanden. **Überprüfung:** Die Dateien aus deinem Test in Übung C sind nur dann enthalten, wenn du sie in `/etc` angelegt hast. Die Testdatei liegt in deinem Arbeitsverzeichnis. **Hinweis:** Verwende für Untersuchungen auf einem Image den Pfad zum eingehängten, schreibgeschützten Dateisystem. Auf einem laufenden System ändert sich die Liste durch normale Betriebsvorgänge.

Übung E: Paketinstallationen auswerten

Lernziel: Paketinstallationen in einem Zeitraum finden. **Voraussetzungen:** Debian oder Ubuntu mit `/var/log/dpkg.log`. **Aufgabenstellung:** Suche nach Installationen in den letzten Wochen, auch in komprimierten Protokollen.

Befehle zu Übung E

```
$ sudo zgrep " install " /var/log/dpkg.log*
```

Erklärung: `zgrep` durchsucht auch komprimierte Dateien, der Platzhalter `*` nimmt alle rotierten Protokolle auf. Die Zeile `install` markiert die Neuinstallation eines Pakets. Jede Zeile enthält Datum, Uhrzeit, Aktion, Paketname und Versionsangaben. **Erwartetes Ergebnis:** Zeilen mit Datum, Paketname und Version für alle Installationen im Protokollzeitraum. **Überprüfung:** Ein Paket, das du in der Übung installiert hast, erscheint mit dem richtigen Zeitstempel. **Fehleranalyse:** Keine Treffer? Die Protokolle könnten rotiert und gelöscht sein. Das ist ein Befund, den du im Protokoll festhältst.

Übung F: Benutzer und geplante Aufgaben prüfen

Lernziel: Konten mit interaktiver Shell und geplante Aufgaben lesen, ohne etwas zu verändern. **Aufgabenstellung:** Liste alle Konten mit einer Shell, die eine Anmeldung erlaubt, und alle systemweiten cron-Einträge.

Befehle zu Übung F

```
$ getent passwd | grep -vE "nologin|false$"
$ ls -la /etc/cron.d
```

```
$ sudo cat /etc/crontab
```

Erklärung: `getent passwd` liest die Kontendatenbank, auch wenn Konten über andere Quellen wie LDAP bereitgestellt werden. `grep -v` blendet Konten aus, die keine interaktive Anmeldung erlauben. `cat` zeigt die systemweite Crontab, `ls -la` listet die Dateien im Verzeichnis `cron.d`. **Erwartetes Ergebnis:** Eine Liste interaktiver Konten, dein Testbenutzer ist darunter, und die bekannten Einträge der Crontab. **Überprüfung:** Alle Einträge, die du nicht erklären kannst, notierst du mit Dateipfad und Zeile als Prüfpunkt. **Sicherheitsbetrachtung:** Diese Übung liest nur. Die Datei `/etc/shadow` lassen wir hier bewusst aus, siehe Abschnitt Benutzer, Gruppen und Anmeldedaten.

Kontrollfragen

1. Welche zwei Pfade oder Quellen enthalten Authentifizierungsereignisse bei Debian oder Ubuntu, und wovon hängt ab, welche vorhanden ist? (leicht)
2. Warum fehlt ein Befehl in der Befehlshistorie, obwohl er ausgeführt wurde? (mittel)
3. Welcher der drei Zeitstempel einer Datei ändert sich, wenn nur die Rechte geändert werden? (leicht)
4. Ein `Accepted publickey`-Eintrag steht im Protokoll. Was belegt er, und was nicht? (mittel)
5. Eine Datei in `/etc/cron.d` hat einen Change-Zeitstempel aus der Nacht des Vorfalls. Welche Schlüsse sind zulässig, welche nicht? (schwer)

Lösungshinweise: 1. `/var/log/auth.log` (bei aktivem rsyslog) und das systemd-Journal. Welche vorhanden ist, hängt davon ab, ob rsyslog installiert und aktiv ist. 2. Die Shell schreibt die Historie erst beim Beenden, eine noch laufende oder nicht sauber beendete Sitzung fehlt deshalb. Auch Einstellungen oder Löschungen können den Eintrag entfernt haben. 3. Der Change-Zeitstempel. Der Modify-Zeitstempel bleibt gleich, weil der Inhalt nicht verändert wurde. 4. Dass sich für das Konto erfolgreich mit einem Schlüssel angemeldet wurde. Nicht belegt ist, wer den Schlüssel besaß, und was danach geschah. 5. Zulässig ist: Die Datei wurde zu diesem Zeitpunkt in

ihren Metadaten verändert. Nicht zulässig ist: Sie wurde von einer bestimmten Person oder zu einem bestimmten Zweck verändert. Dafür braucht es den Abgleich mit Protokollen und dem Inhalt der Datei.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

[ ] die Protokollquellen eines Systems prüfen und die passende Quelle auswählen kannst,

[ ] Anmeldeereignisse aus dem Journal oder aus auth.log filtern kannst,

[ ] die drei Zeitstempel einer Datei unterscheiden kannst,

[ ] angeben kannst, was eine Befehlshistorie, ein Anmeldeeintrag und ein Metadatenzeitstempel belegen und was nicht.

Zusammenfassung

- Die Protokollquelle hängt von Distribution und Konfiguration ab. Prüfe zuerst, welche Quellen vorhanden sind.
- Das systemd-Journal speichert Ereignisse, die auf reinen Journal-Systemen die einzige Protokollquelle sind.
- Anmeldeereignisse belegen Konten, nicht Personen. Die Befehlshistorie ist eine Hinweisquelle.
- Metadaten sind leicht veränderbar. Zeitstempel brauchen den Abgleich mit anderen Quellen.

Kapitel 8: Timeline-Analyse

Lernziele

- Du erklärst, warum alle Zeitangaben einer Untersuchung auf dieselbe Zeitbasis gebracht werden müssen.
- Du erstellst eine einfache Timeline aus Dateizeitstempeln und Protokollereignissen.
- Du erkennst die Grenzen von Zeitangaben: Zeitzonen, Sommerzeit, Uhrabweichungen und manipulierbare Stempel.

Gut zu wissen

Dieses Kapitel verwendet eine bewusst einfache Methode: Zeitstempel werden aus den Quellen gelesen, in UTC umgewandelt und sortiert. Ein spezialisiertes Werkzeug wie Plaso kann dasselbe in größerem Umfang leisten. Ob es für deine Untersuchung nötig ist, hängt von der Datenmenge ab.

Was eine Timeline ist

Eine forensische Timeline ist eine zeitlich sortierte Liste von Ereignissen aus verschiedenen Quellen. Sie beantwortet die Frage, was in welcher Reihenfolge geschah. Einzelne Zeitangaben sagen wenig, eine Folge von Ereignissen aus Protokollen, Dateien und Registry kann dagegen einen Ablauf erkennen lassen: eine Anmeldung, das Starten eines Programms, das Anlegen einer Datei und danach eine Netzwerkverbindung.

Diagramm 8.1: Aufbau einer forensischen Timeline

```
Quellen, jeweils nach UTC umgerechnet:
  Dateizeitstempel (stat)
  Ereignisprotokolle (Windows)
  Journal (journalctl)

Ergebnis, nach Zeit sortiert, jede Zeile mit Quelle:
  2026-11-10T13:58:02Z  DATEI    Datei X geändert
  2026-11-10T14:02:10Z  WINDOWS  Anmeldung Konto Y
  2026-11-10T14:03:44Z  JOURNAL  Dienst Z gestartet
```

Jede Zeile der Timeline muss ihre Quelle nennen. Ohne Quelle lässt sich später nicht prüfen, ob ein Ereignis aus Protokoll, Dateisystem oder Registry stammt, und das ist für die Bewertung entscheidend.

Die vier Zeitstempel einer Datei

Dateisysteme speichern für jede Datei mehrere Zeitstempel. Unter Linux sind es Access, Modify, Change und, je nach Dateisystem, Birth (Kapitel 7). Unter NTFS, dem Dateisystem von Windows, gibt es ebenfalls vier Zeiten: Erstellung, letzte Änderung des Inhalts, letzte Änderung des Eintrags in der Master File Table und letzter Zugriff. Die Bezeichnungen sind verschieden, die Idee ist dieselbe.

Die Kurzform MACB (Modified, Accessed, Changed, Birth) hilft, diese Zeiten in einer Timeline zu unterscheiden. Ein Zugriff ist etwas anderes als eine Änderung, und eine Änderung der Metadaten ist etwas anderes als eine Änderung des Inhalts.

> **WARNUNG – Sicherheitsrisiko**
>
> Zeitstempel sind leicht zu verändern, sowohl unbeabsichtigt durch Kopieren oder Entpacken als auch absichtlich mit Werkzeugen, die Zeiten setzen. Unter Linux kann etwa `touch` mit einer Datumsangabe Zeitstempel überschreiben. Eine Timeline, die sich auf einen einzelnen Stempel stützt, ist deshalb schwach. Belastbar wird sie erst durch Abgleich mit unabhängigen Quellen.

Zeitzonen, UTC und lokale Zeit

Der häufigste Fehler in Timelines sind verschobene Zeiten. Windows-Ereignisprotokolle speichern Zeitstempel in UTC, die Ereignisanzeige zeigt sie in lokaler Zeit. Das systemd-Journal speichert in UTC und zeigt standardmäßig lokale Zeit an. Dateizeitstempel werden je nach Werkzeug in UTC oder lokal angezeigt. Wer diese Anzeigen ungeprüft mischt, erhält zwei Ereignisse, die scheinbar eine Stunde auseinanderliegen, obwohl sie gleichzeitig stattfanden.

Die Regel dieses Buches ist einfach: Alle Zeiten werden vor dem Vergleich nach UTC umgerechnet und mit dem Zusatz Z oder +00:00 gekennzeichnet. Lokale Zeit ist nur im Bericht erlaubt, und dann mit Zeitzone angegeben.

Sommer- und Winterzeit

Die Umstellung auf Winterzeit kann Ereignisse mehrdeutig machen. In Deutschland endet die Sommerzeit am letzten Sonntag im Oktober um 03:00 Uhr Ortszeit, die Uhr springt dann auf 02:00 zurück. Die Zeit von 02:00 bis 03:00 kommt also zweimal vor. Am 25. Oktober 2026 zum Beispiel gibt es eine Stunde, in der eine lokale Zeitangabe ohne Zeitzone zwei verschiedene Ereignisse bezeichnen kann. Eine Angabe in UTC ist eindeutig, deshalb ist sie für die Timeline die richtige Basis.

Uhrabweichungen und NTP

Jeder Rechner hat eine eigene Uhr, und jede Uhr driftet. In der Praxis synchronisieren sich Systeme über NTP (Network Time Protocol) mit einer Zeitquelle. Bleibt die Uhr eines Systems über längere Zeit ohne Synchronisation, kann sich eine Abweichung von Sekunden oder Minuten ergeben. Für eine Timeline ist das relevant, wenn zwei Ereignisse auf verschiedenen Systemen korreliert werden sollen.

Prüfe deshalb, ob die Uhr des untersuchten Systems synchronisiert war und welche Abweichung zum Zeitpunkt der Sicherung bestand. Unter Linux liefert `timedatectl status` Angaben zur Zeitzone und dazu, ob die Systemuhr synchronisiert ist. Unter Windows zeigt `w32tm /query /status` den Status des Zeitdienstes an. Halte den Befund im Protokoll fest, auch wenn er unauffällig ist.

Ereigniskorrelation

Ein Ereignis wird wichtiger, wenn es durch eine zweite, unabhängige Quelle bestätigt wird. Eine Dateiänderung um 14:03 UTC in einem Verzeichnis, eine Anmeldung um 14:02 UTC und ein Protokoll mit dem Start eines Programms um 14:03 UTC bilden zusammen einen Zusammenhang. Ein einzelnes der drei Ereignisse wäre dagegen nur ein Hinweis. Die Stärke einer Timeline liegt in dieser Mehrfachbestätigung.

Die Korrelation hat aber Grenzen. Zwei Ereignisse zur gleichen Zeit beweisen keinen Zusammenhang, wenn sie von unterschiedlichen Prozessen ausgelöst wurden. Die Timeline zeigt, was zeitlich zusammenfällt, die Schlussfolgerung über Ursache und Wirkung ist eine Interpretation.

Grenzen der Timeline-Analyse

Eine Timeline kann nur so vollständig sein wie ihre Quellen. Protokolle, die nicht aktiv waren, Journale, die rotiert wurden, und Dateien, die gelöscht wurden, erscheinen nicht. Eine Lücke in der Timeline ist deshalb kein Beleg dafür, dass dort nichts geschah. Dokumentiere, welche Quellen geprüft wurden und welche Zeiträume sie abdecken.

Praxis: Eine Timeline erstellen

Für die Übungen brauchst du eine kleine Testmenge: ein Verzeichnis mit einigen Dateien, die du selbst angelegt und verändert hast, einen Benutzer mit Anmeldungen in der VM und, für den Windows-Teil, eine Windows-11-VM mit einer Datei und einer Änderung. Lege wie in den vorigen Kapiteln einen Snapshot an.

Übung A: Zeitzonen prüfen und vergleichen

Lernziel: die lokale Zeitzone und den UTC-Stand eines Systems feststellen, bevor Zeiten verglichen werden. **Voraussetzungen:** Linux-Testsystem. **Aufgabenstellung:** Lies die aktuelle Zeit in UTC und lokaler Zeit sowie den Status der Zeitsynchronisation aus.

Befehle zu Übung A

```
$ date -u
$ date
```

```
$ timedatectl status
```

Erklärung: `date -u` zeigt UTC, `date` ohne Option die lokale Zeit mit Zeitzone. `timedatectl status` enthält die Zeile „Time zone“ und den Eintrag „System clock synchronized“. **Erwartetes Ergebnis:** Zwei Zeitangaben, die sich um den Versatz der lokalen Zeitzone unterscheiden, und die Bestätigung, ob die Uhr synchronisiert ist. **Überprüfung:** Die Differenz zwischen beiden Angaben entspricht dem Zeitzonenversatz aus `timedatectl`. **Fehleranalyse:** Die Uhr ist nicht synchronisiert? Notiere das. Für die Timeline musst du dann mit einer möglichen Abweichung rechnen.

Übung B: Dateizeitstempel in UTC sammeln

Lernziel: alle drei Zeitstempel einer Dateigruppe in UTC lesen und sortieren. **Voraussetzungen:** Verzeichnis `testdaten` mit mindestens drei Dateien, von denen du eine später veränderst. **Aufgabenstellung:** Lies für alle Dateien in `testdaten` die Zeitstempel in UTC aus und sortiere sie nach Änderungszeit.

Befehle zu Übung B

```
$ TZ=UTC stat -c '%y | %x | %z | %n' testdaten/* | sort
```

Erklärung: `TZ=UTC` setzt die Anzeige für diesen Aufruf auf UTC. `stat -c` gibt ein eigenes Format aus: `%y` Änderung des Inhalts, `%x` letzter Zugriff, `%z` Änderung der Metadaten, `%n` Dateiname. Die Spalten folgen der Aufzählung oben: Änderung des Inhalts, Zugriff, Änderung der Metadaten, Name. Die Ausgabe ist durch `sort` sortiert. **Erwartetes Ergebnis:** Eine Zeile pro Datei mit drei Zeitstempeln und dem Namen. **Überprüfung:** Ändere eine Datei danach mit `echo "Zusatz" >> testdaten/datei2.txt` und führe den Befehl erneut aus. Die Änderungszeit der Datei muss neuer sein als vorher. **Fehleranalyse:** Die Zeiten passen nicht zu deiner lokalen Uhr? Prüfe mit Übung A die Zeitzone. Sortierung wirkt falsch? `sort` vergleicht Text, und bei unterschiedlichen Format-Varianten (etwa mit und ohne Sekundenbruchteile) kann die Reihenfolge abweichen.

Übung C: Journal-Ereignisse in ISO-Format für die Timeline exportieren

Lernziel: Protokollereignisse in ein Format bringen, das sich mit Dateizeiten zusammenführen lässt. **Voraussetzungen:** Linux-Testsystem mit Journal, mindestens eine Anmeldung oder ein Dienststart in den letzten Stunden. **Aufgabenstellung:** Exportiere die Journalereignisse der letzten Stunde mit ISO-Zeitstempeln in UTC.

Befehle zu Übung C

```
$ journalctl --utc --since "1 hour ago" -o short-iso --no-pager >
    journal_export.txt
$ head -n 5 journal_export.txt
```

Erklärung: `-o short-iso` schreibt jeden Zeitstempel im Format `JJJJ-MM-TT HH:MM:SS` mit Zeitzonenkennzeichnung. Die Ausgabe wird in eine Datei geschrieben, damit du sie mit den Dateizeiten aus Übung B zusammenführen kannst. `--utc` sorgt dafür, dass die Zeiten in UTC stehen. **Erwartetes Ergebnis:** Eine Textdatei, deren Zeilen mit einem ISO-Zeitstempel beginnen. **Überprüfung:** Der Zeitstempel einer Anmeldung aus deiner Übung steht in der Datei und stimmt mit der Ausgabe in Übung A überein. **Fehleranalyse:** Die Datei ist leer? Prüfe, ob das Journal die Zeitspanne abdeckt, mit `journalctl --disk-usage`.

Übung D: Windows-Ereignisse und Dateizeiten in UTC umwandeln

Lernziel: die Zeitstempel einer Windows-Datei und eines Ereignisses auf UTC bringen und vergleichen. **Voraussetzungen:** Windows-11-VM mit einer Testdatei und einer Anmeldung. **Aufgabenstellung:** Lies die UTC-Zeit einer Testdatei und die UTC-Zeit der letzten Anmeldung aus.

Befehle zu Übung D

```
PS> Get-Item .\testdatei.txt | Select-Object Name,
    LastWriteTimeUtc, CreationTimeUtc
PS> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624}
    -MaxEvents 1 |
    ForEach-Object { $_.TimeCreated.ToUniversalTime() }
```

Erklärung: Die Eigenschaften `LastWriteTimeUtc` und `CreationTimeUtc` liefern die Zeiten bereits in UTC. Bei Ereignissen ist `TimeCreated` in lokaler Zeit, `ToUniversalTime()` rechnet sie um. **Erwartetes Ergebnis:** Zwei UTC-Angaben, die sich direkt vergleichen lassen. **Überprüfung:** Die Anmeldung muss nach der Erstellung der Testdatei liegen, wenn du die Datei nach deiner letzten Anmeldung angelegt hast. Ist das nicht der Fall, prüfe die Reihenfolge deiner Schritte. **Fehleranalyse:** Eine Stunde Differenz trotz gleicher Uhr? Prüfe, ob einer der Werte in lokaler Zeit eingetragen wurde.

Übung E: Timeline zusammenführen

Lernziel: Dateien- und Protokollereignisse in einer gemeinsamen, chronologisch sortierten Liste zusammenführen. **Voraussetzungen:** die Dateien aus Übung B und C im selben Arbeitsverzeichnis. **Aufgabenstellung:** Bringe beide Quellen auf ein gemeinsames Zeitformat, kennzeichne jede Zeile mit ihrer Quelle und sortiere sie.

Befehle zu Übung E

```
$ for f in testdaten/*; do echo "$(date -u -d "@$(stat -c %Y
    "$f")" +%Y-%m-%dT%H:%M:%SZ) DATEI $f"; done >
    quelle_dateien.txt
$ sed -E 's/^(.{19})[^ ]* /\1Z JOURNAL /' journal_export.txt >
    quelle_journal.txt
$ sort quelle_dateien.txt quelle_journal.txt > timeline.txt
```

Erklärung: Die erste Zeile liest für jede Datei den Änderungszeitpunkt als Sekundenzahl, wandelt ihn mit `date -u` in UTC um und setzt das Format `JJJJ-MM-TTThh:mm:ssZ` an den Anfang der Zeile. Die zweite Zeile kürzt die Zeitstempel des Journalexports auf dasselbe Format und hängt ein Z an. Erst danach beginnen beide Quellen mit demselben Zeitformat. Die Quellenangabe steht hinter dem Zeitstempel, damit das Sortieren nach der Zeit erfolgt und nicht nach der Quelle. **Erwartetes Ergebnis:** Eine Datei `timeline.txt`, in der Dateiänderungen und Journalereignisse nach Zeit geordnet erscheinen. Jede Zeile beginnt mit dem Zeitstempel und nennt danach DATEI oder JOURNAL. **Überprüfung:** Deine Dateiänderung aus Übung B und deine Anmeldung aus Übung C müssen beide darin auftauchen, in der richtigen Reihenfolge. Prüfe außerdem, ob die Zeilen mit Z in derselben Reihenfolge stehen wie ihre Zeitstempel. **Fehleranalyse:** Die

Reihenfolge stimmt nicht? Prüfe mit `head -n 3 journal_export.txt`, ob die Zeitstempel das erwartete Format haben. Die Sed-Zeile funktioniert nur, wenn der Export mit `-o short-iso` erzeugt wurde. Die Zeitangaben aus dem Dateisystem sind auf Sekunden genau; Sekundenbruchteile gehen hier verloren, das ist für diese Übung ausreichend.

Übung F (optional): Timeline mit Plaso

Lernziel: ein spezialisiertes Werkzeug auf eine kleine Testmenge anwenden und mit der manuellen Timeline vergleichen. **Voraussetzungen:** Plaso in einer Version, die du aus der Dokumentation des Projekts installierst. Die Optionen unterscheiden sich zwischen Versionen, prüfe sie mit `log2timeline.py --help` vor dem Einsatz. **Aufgabenstellung:** Erstelle eine Timeline des Testverzeichnisses und exportiere sie als CSV.

Befehle zu Übung F (Version prüfen)

```
$ log2timeline.py --storage-file timeline.plaso testdaten/
$ psort.py -o l2tcsv -w timeline_plaso.csv timeline.plaso
```

Erklärung: `log2timeline.py` liest die Quellen und schreibt eine Speicherdatei. `psort.py` sortiert und exportiert sie. Das Format `l2tcsv` ist ein CSV mit Zeitstempeln, Quelle und Beschreibung. **Erwartetes Ergebnis:** Eine CSV-Datei mit mehr Zeilen als deine manuelle Timeline, weil Plaso zusätzliche Metadaten auswertet. **Überprüfung:** Vergleiche die Zeilen zu deiner Testdatei mit den Ergebnissen aus Übung B. Die Zeitstempel müssen übereinstimmen, wenn du die Zeitzone im Ausgabeformat beachtest. **Fehleranalyse:** Die Optionen werden abgelehnt? Prüfe die Version und die Hilfe des Befehls. Plaso ist für große Datenmengen gedacht und kann auf einer kleinen Testmenge übertrieben wirken; das ist beabsichtigt, weil du die Ausgabe prüfen sollst.

Typische Fehler

- Zeiten aus Protokollen und Dateien werden ohne Umrechnung gemischt.
- Eine lokale Zeitangabe ohne Zeitzone wird bei der Sommerzeitumstellung falsch zugeordnet.
- Eine Uhr ohne Synchronisation wird als korrekt behandelt.
- Die Timeline enthält keine Quellenangabe pro Zeile.

• Eine Lücke wird als Beleg für das Fehlen von Ereignissen gewertet.

> **Zusammenfassung**
>
> • Eine Timeline ordnet Ereignisse aus verschiedenen Quellen nach Zeit. Jede Zeile nennt ihre Quelle.
> • Alle Zeiten werden vor dem Vergleich in UTC umgerechnet. Die Sommerzeitumstellung macht lokale Angaben mehrdeutig.
> • Die Uhrsynchronisation des untersuchten Systems gehört in das Protokoll.
> • Zeitstempel sind leicht veränderbar. Belastbar wird eine Timeline durch unabhängige Bestätigung.

Kontrollfragen

1. Warum werden alle Zeiten vor dem Vergleich in UTC umgerechnet? (leicht)
2. Was bedeutet MACB, und warum unterscheidet man die vier Zeitstempel? (leicht)
3. Warum ist die Stunde beim Wechsel zur Winterzeit in lokaler Zeit problematisch? (mittel)
4. Eine Dateiänderung und eine Anmeldung liegen zeitlich nah beieinander. Was belegt das, und was nicht? (mittel)
5. Eine Timeline zeigt eine Lücke von zwei Stunden in den Journaldaten eines Servers. Welche Erklärungen sind möglich, und wie prüfst du sie? (schwer)

Lösungshinweise: 1. Weil Quellen Zeiten in unterschiedlicher Zeitzone speichern oder anzeigen, und nur UTC einen eindeutigen Vergleich erlaubt. 2. Modify, Accessed, Changed, Birth. Die Zeiten beschreiben unterschiedliche Vorgänge: Inhaltsänderung, Zugriff, Metadatenänderung und Erstellung. 3. In der Umstellungsnacht kommt eine Stunde zweimal vor, eine lokale Angabe ohne Zeitzone ist dann nicht eindeutig. 4. Zeitliche Nähe ist ein Hinweis auf einen möglichen Zusammenhang. Sie belegt weder, dass die Anmeldung die Änderung verursacht hat, noch wer sie vorgenommen hat. 5. Mögliche Ursachen sind ein Ausfall des Journals, eine Rotation oder Löschung, eine nicht laufende Protokollierung oder eine Zeitumstellung. Prüfen lässt sich das mit `journalctl --list-boots` und `journalctl`

`--disk-usage`, mit den Angaben zur Uhrsynchronisation und mit anderen Quellen für denselben Zeitraum.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] Zeiten aus verschiedenen Quellen in UTC umrechnest,
- [ ] eine einfache Timeline mit Quellenangabe pro Zeile erstellst,
- [ ] die Grenzen von Sommerzeit, Uhrabweichungen und manipulierbaren Stempeln erklären kannst,
- [ ] eine Lücke in einer Timeline als möglichen Befund und nicht als Beweis einordnest.

Kapitel 9: Digitale Netzwerkforensik

Lernziele

- Du erklärst, welche Spuren Netzwerkverkehr hinterlässt und welche davon sich auch bei verschlüsselter Kommunikation lesen lassen.
- Du filterst und analysierst einen Mitschnitt mit Wireshark und tshark.
- Du benennst, was eine IP-Adresse, ein DNS-Eintrag oder ein TLS-Hostname belegt und was nicht.

WARNUNG – Sicherheitsrisiko

Mitschnitte dürfen nur von Netzen erstellt werden, für die du berechtigt bist. Die Übungen in diesem Kapitel laufen vollständig innerhalb deiner eigenen VM über die Loopback-Schnittstelle. Das unbefugte Abfangen fremder Kommunikation kann strafbar sein. Mitschnitte dürfen nur im Rahmen einer entsprechenden Berechtigung erstellt werden. Sie können personenbezogene Daten enthalten; deren Speicherung und Weitergabe müssen datenschutzrechtlich geprüft werden.

Was Netzwerkspuren sind

Ein Netzwerkmitschnitt, auch PCAP-Datei genannt, enthält die Pakete, die über eine Schnittstelle gelaufen sind, mit Zeitstempel. Das Dateiformat stammt aus der Bibliothek libpcap, daneben gibt es das Format pcapng. Wireshark und tshark lesen beide.

Netzwerkspuren entstehen an mehreren Stellen, und jede Stelle sieht etwas anderes. Der Mitschnitt auf einem Endgerät zeigt den Verkehr dieses Geräts. Firewall, Router und Proxy protokollieren Verbindungen, die durch sie laufen. DNS-Server speichern Anfragen, je nach Konfiguration. Ein Mitschnitt zeigt meist, was unverschlüsselt sichtbar ist, und ein Protokoll zeigt, was der Dienst selbst festgehalten hat. Beides ergänzt sich, ersetzt sich aber nicht.

Diagramm 9.1: Netzwerkforensischer Analyseprozess

```
1. Quelle festlegen (Schnittstelle, Gerät, Zeitraum, Berechtigung)
2. Mitschnitt erstellen und Hashwert bilden (Kapitel 5)
3. Protokolle der Netzwerkkomponenten sichern
4. Mitschnitt filtern: Protokoll, Adresse, Port, Zeitraum
5. Auffälligkeiten einzeln prüfen: DNS, HTTP, TLS-Hostnamen, Verbindungsdauer
6. Ergebnisse mit Timeline und anderen Quellen abgleichen (Kapitel 8)
7. Befund dokumentieren, Aussagegrenzen festhalten
```

IP-Adressen, Ports und Protokolle

Eine IP-Adresse bezeichnet ein Netzwerkinterface, nicht eine Person. Hinter einer öffentlichen Adresse können viele Geräte stehen, etwa bei Netzwerkadressübersetzung (NAT) in einem Heimnetz oder Unternehmensnetz. Außerdem vergibt ein DHCP-Server Adressen nur für begrenzte Zeit. Um von einer Adresse auf ein Gerät zu schließen, braucht man die Zuordnung zum Zeitpunkt des Verkehrs, meist aus den Protokollen des DHCP-Servers.

Ein Port ordnet eine Verbindung einem Dienst zu, zum Beispiel 80 für HTTP oder 443 für HTTPS. Diese Zuordnung ist eine Konvention, kein Beweis: Ein Dienst kann auf jedem Port laufen, und Schadsoftware nutzt bewusst Standardports.

DNS: Namensauflösung als Spur

Bevor ein Gerät eine Verbindung zu einem Namen wie `example.com` aufbaut, fragt es per DNS nach der zugehörigen Adresse. Diese Anfrage steht unverschlüsselt im Mitschnitt, sofern kein DNS über TLS oder HTTPS genutzt wird, und sie enthält den Namen, nach dem gefragt wurde. Auch dann, wenn die Verbindung selbst verschlüsselt ist, zeigt der DNS-Verkehr also, welche Dienste ein Gerät ansteuern wollte.

Eine DNS-Anfrage belegt, dass ein Gerät den Namen aufgelöst hat. Sie belegt nicht, dass danach eine Verbindung zustande kam, und nicht, dass ein Nutzer die Seite aufgerufen hat. Browser, Betriebssysteme und Dienste senden im Hintergrund eigene Anfragen, und Antworten werden zwischengespeichert.

HTTP und TLS

Unverschlüsseltes HTTP zeigt im Mitschnitt die Methode, den Pfad, den Host-Namen und oft auch Antwortinhalte. Bei HTTPS sind Pfad und Inhalt verschlüsselt. Der Name des Ziels steht aber in der TLS-Begrüßung, im Feld Server Name Indication (SNI). Dieser Hostname ist daher auch bei verschlüsselten Verbindungen meist sichtbar. Ob eine Verbindung zu einem Namen gehörte, lässt sich deshalb oft klären, auch ohne den Inhalt zu kennen.

Die Grundlagen von TLS sind im Kryptographie-Handbuch dieser Reihe ausführlich behandelt. Hier genügt: Der Handshake ist sichtbar, der Inhalt danach nicht.

Verbindungen und Dauer

Ein Mitschnitt zeigt für jede TCP-Verbindung den Beginn, das Ende, die übertragene Datenmenge und die beteiligten Adressen und Ports. Auffällig können Verbindungen sein, die sehr lange dauern, regelmäßig in gleichen Abständen stattfinden oder ungewöhnlich viele Daten übertragen. Solche Muster sind Anlass für eine genauere Prüfung, aber für sich genommen keine Beweise für Datenabfluss.

Grenzen der Netzwerkforensik

Ein Mitschnitt ist immer nur das, was zu diesem Zeitpunkt auf dieser Schnittstelle sichtbar war. Pakete, die nicht auf dieser Schnittstelle liefen, fehlen. Ein Mitschnitt kann durch Paketverlust unvollständig sein, und ein Mitschnitt auf einem Gerät zeigt nicht den Verkehr eines anderen Geräts im selben Netz, sofern er nicht über einen Switch mit Spiegelport aufgenommen wurde. Ohne Protokolle der Netzkomponenten bleiben viele Fragen offen.

Typische Fehler

- Eine IP-Adresse wird einer Person zugeordnet, ohne die Zuordnung zum Zeitpunkt zu prüfen.
- Eine DNS-Anfrage wird als Beleg für einen Seitenaufruf gewertet.
- Der Mitschnitt wird auf der falschen Schnittstelle erstellt, etwa dem Loopback statt dem Netzwerkadapter.

• Der Hashwert des Mitschnitts wird nicht gebildet.

• Verschlüsselter Verkehr wird als Beleg für Inhalte gewertet, die nicht sichtbar sind.

> **Zusammenfassung**
>
> • Netzwerkspuren entstehen auf mehreren Stellen, jede sieht einen anderen Ausschnitt.
> • DNS-Anfragen und TLS-Hostnamen zeigen Ziele auch bei verschlüsselter Kommunikation.
> • IP-Adressen und Ports belegen Verbindungen, nicht Personen oder Inhalte.
> • Jeder Mitschnitt braucht einen Hashwert, eine dokumentierte Schnittstelle und eine Aussagegrenze.

Praxis: Netzwerkspuren in einem eigenen Testmitschnitt

Die Übungen laufen auf einer Linux-VM. Für die Übungen A bis C brauchst du keinen Internetzugang, weil der Verkehr über die Loopback-Schnittstelle `lo` bleibt. Für Übung D und E brauchst du Internetzugang und kannst die Beispieldomäne `example.com` verwenden. Installiere Wireshark mit tshark und `tcpdump` über die Paketverwaltung deiner Distribution. Lege vorher einen Snapshot der VM an.

Übung A: Testmitschnitt über die Loopback-Schnittstelle erstellen

Lernziel: einen kontrollierten HTTP-Verkehr erzeugen und mitschneiden, und den Mitschnitt mit einem Hashwert sichern. **Voraussetzungen:** Python 3, tcpdump, curl. **Aufgabenstellung:** Starte einen einfachen Webserver auf dem Testrechner, schneide den Verkehr zu diesem Server mit und sichere den Mitschnitt.

Befehle zu Übung A

```
$ mkdir -p netzlabor && cd netzlabor
$ echo "Testinhalt" > test.txt
$ python3 -m http.server 8080 --bind 127.0.0.1 & echo $! > srv.pid
```

```
$ sudo tcpdump -i lo -w lokal.pcap port 8080 & echo $! > tcp.pid
$ sleep 2
$ curl -s http://127.0.0.1:8080/test.txt
$ sudo kill $(cat tcp.pid)
$ kill $(cat srv.pid)
$ sha256sum lokal.pcap > lokal.pcap.sha256
```

Erklärung: `python3 -m http.server` startet einen Webserver nur auf `127.0.0.1`, damit nichts von außen erreichbar ist. `tcpdump -i lo` schneidet die Loopback-Schnittstelle mit, `-w` schreibt in eine Datei, der Filter `port 8080` begrenzt auf diesen Verkehr. `curl` erzeugt die Anfrage. Die Prozess-IDs der beiden Hintergrundprozesse landen in Dateien (`echo $!`), damit `kill` sie zuverlässig beenden kann. Das `sleep 2` gibt tcpdump Zeit, bevor die Anfrage kommt. Die letzte Zeile bildet den Hashwert, wie in Kapitel 5 beschrieben. **Erwartetes Ergebnis:** Eine Datei `lokal.pcap` mit einigen Paketen, dazu die Hashdatei. **Überprüfung:** `sha256sum -c lokal.pcap.sha256` meldet `OK`. **Fehleranalyse:** Der Port ist belegt? Wähle einen anderen Port und passe Server und Filter an. Die Datei bleibt leer? Prüfe, ob `curl` tatsächlich auf 127.0.0.1 zugreift und ob der Server läuft. **Sicherheitsbetrachtung:** Der Server ist nur lokal erreichbar, beende ihn nach der Übung.

Übung B: HTTP-Anfragen mit tshark filtern

Lernziel: die Anfrage aus Übung A im Mitschnitt finden und die relevanten Felder auslesen. **Voraussetzungen:** Übung A abgeschlossen, tshark installiert. **Aufgabenstellung:** Lies alle HTTP-Anfragen aus `lokal.pcap` mit Zeitstempel, Quelle, Methode und Pfad aus.

Befehle zu Übung B

```
$ tshark -r lokal.pcap -Y http.request -T fields -E header=y -E
    separator=, \
    -e frame.time -e ip.src -e http.request.method -e
        http.request.uri
```

Erklärung: `-r` liest die Datei, `-Y` setzt den Anzeigefilter `http.request`, der nur HTTP-Anfragen zeigt. `-T fields` gibt einzelne Felder aus, `-e` wählt die Felder. `-E header=y` schreibt eine Kopfzeile, `-E separator=,` trennt die Felder durch Kommas. `frame.time` ist der Zeitstempel des Pakets in der

lokalen Zeitzone. **Erwartetes Ergebnis:** Eine Zeile mit dem Aufruf von `/test.txt`, als Quelladresse `127.0.0.1`. **Überprüfung:** Der Zeitstempel stimmt mit dem Zeitpunkt deines `curl`-Aufrufs überein. Wandle ihn für die Timeline nach UTC um (Kapitel 8). **Fehleranalyse:** Keine Zeile? Der Filter greift nur, wenn der Mitschnitt HTTP im Klartext enthält. Prüfe mit `tshark -r lokal.pcap -c 5`, welche Pakete überhaupt vorhanden sind. **Hinweis:** Die Option `-t` steuert das Zeitformat. Ob `-t ud` die UTC-Ausgabe liefert, prüfst du mit `tshark --help` für deine Version.

Übung C: Eine TCP-Verbindung rekonstruieren

Lernziel: den Inhalt einer Verbindung aus den Paketen zusammensetzen. **Voraussetzungen:** Übung A abgeschlossen. **Aufgabenstellung:** Rekonstruiere den Verkehr der Verbindung aus Übung A als Text. Die Stream-Nummer findest du in Übung B über das Feld `tcp.stream`.

Befehle zu Übung C

```
$ tshark -r lokal.pcap -q -z follow,tcp,ascii,0
```

Erklärung: `-z follow,tcp,ascii,0` rekonstruiert den TCP-Stream mit der Nummer 0 als Text. Falls die Stream-Nummer des HTTP-Verkehrs nicht 0 ist, ersetze sie durch den Wert aus `tcp.stream`. Die Ausgabe zeigt Anfrage und Antwort in der Reihenfolge, in der sie übertragen wurden. **Erwartetes Ergebnis:** Der Text der HTTP-Anfrage, gefolgt von der Antwort mit `Testinhalt`. **Überprüfung:** Der Inhalt der Antwort stimmt mit dem Inhalt von `test.txt` überein. **Fehleranalyse:** Der Stream ist leer? Die Nummer stimmt nicht. Prüfe sie in Übung B mit `-e tcp.stream`.

Übung D: DNS-Anfragen mitschneiden und auswerten

Lernziel: erkennen, welche Namen ein Gerät auflösen wollte, unabhängig davon, ob die Verbindung danach zustande kam. **Voraussetzungen:** Internetzugang, dig (Paket dnsutils oder bind-utils) oder ein beliebiger Namensauflösungsbefehl. **Aufgabenstellung:** Schneide DNS-Verkehr auf allen Schnittstellen mit, löse `example.com` auf und werte die Anfragen aus.

Befehle zu Übung D

```
$ sudo tcpdump -i any -w dns.pcap port 53 & echo $! > dns.pid
$ sleep 2
```

```
$ dig +short example.com
$ sudo kill $(cat dns.pid)
$ tshark -r dns.pcap -Y "dns.flags.response == 0" -T fields -E
    header=y \
    -e frame.time -e dns.qry.name
```

Erklärung: `-i any` schneidet alle Schnittstellen mit, `port 53` begrenzt auf DNS. `dig +short` gibt nur die Antwort aus. `dns.flags.response == 0` filtert auf Anfragen, nicht auf Antworten. `dns.qry.name` ist der gefragte Name. **Erwartetes Ergebnis:** Eine Zeile mit `example.com` und dem Zeitstempel der Anfrage. **Überprüfung:** Der Name stimmt mit dem überein, den du aufgelöst hast. Beachte: Ein lokaler Resolver mit Cache sendet möglicherweise keine neue Anfrage, wenn der Name bereits gespeichert ist. Dann fehlt die Zeile, und das muss im Protokoll stehen. **Fehleranalyse:** Keine Anfrage im Mitschnitt? Prüfe mit `resolvectl status` (systemd-resolved) oder `cat /etc/resolv.conf`, über welchen Resolver die Anfrage läuft, und ob ein Cache aktiv ist.

Übung E: TLS-Hostnamen aus dem Handshake lesen

Lernziel: erkennen, dass der Zielname einer HTTPS-Verbindung auch bei verschlüsseltem Inhalt sichtbar ist. **Voraussetzungen:** Internetzugang, curl. **Aufgabenstellung:** Schneide eine HTTPS-Verbindung zu `example.com` mit und lies den Hostnamen aus dem TLS-Handshake aus.

Befehle zu Übung E

```
$ sudo tcpdump -i any -w tls.pcap port 443 & echo $! > tls.pid
$ sleep 2
$ curl -s -o /dev/null https://example.com
$ sudo kill $(cat tls.pid)
$ tshark -r tls.pcap -Y "tls.handshake.type == 1" -T fields -E
    header=y \
    -e frame.time -e tls.handshake.extensions_server_name
```

Erklärung: `tls.handshake.type == 1` filtert auf die ClientHello-Nachricht, die der Client zu Beginn schickt. Das Feld `tls.handshake.extensions_server_name` enthält den SNI-Hostnamen. Der Inhalt der Seite ist verschlüsselt und bleibt unlesbar. **Erwartetes Ergebnis:** Eine Zeile mit `example.com`. Die Antwort und der Seiteninhalt sind im Mitschnitt nicht lesbar. **Überprüfung:** Versuche, mit

`Follow TCP Stream` in der Wireshark-Oberfläche den Inhalt zu lesen. Er muss unleserlich bleiben, der Hostname dagegen nicht. **Fehleranalyse:** Kein SNI im Ergebnis? Der Client sendet ihn nicht, oder die Verbindung lief über einen Proxy. Notiere das.

Übung F: Befund mit Aussagegrenze dokumentieren

Lernziel: die Ergebnisse aus A bis E so festhalten, dass ihr Umfang klar bleibt. **Aufgabenstellung:** Schreibe ein Protokoll für den Mitschnitt aus Übung E. Nenne Quelle, Zeitraum, Schnittstelle, Hashwert, Filter, Ergebnis und Aussagegrenze. **Erwartetes Ergebnis:** Ein Protokoll, in dem steht, dass der Client zu einem Zeitpunkt eine HTTPS-Verbindung zu `example.com` aufgebaut hat, und nicht, welche Inhalte übertragen wurden. **Fehleranalyse:** Die Formulierung enthält „hat Seite X besucht“? Das geht über den Befund hinaus. Ersetze es durch die nachweisbare Verbindung zum Namen.

Kontrollfragen

1. Was zeigt ein Netzwerkmitschnitt, wenn die Verbindung HTTPS verwendet? (leicht)
2. Was belegt eine DNS-Anfrage, und was nicht? (leicht)
3. Warum ist eine IP-Adresse allein kein Beleg für eine bestimmte Person? (mittel)
4. Welcher Filter zeigt im Mitschnitt nur die ClientHello-Nachrichten der TLS-Verbindungen? (mittel)
5. In einem Mitschnitt fehlt die DNS-Anfrage für einen Dienst, obwohl die Verbindung zu ihm nachweisbar ist. Welche Erklärungen sind möglich, und wie prüfst du sie? (schwer)

Lösungshinweise: 1. Den Verbindungsaufbau, den Zielnamen über SNI und die Verbindungsdaten, aber nicht den verschlüsselten Inhalt. 2. Dass ein Gerät einen Namen auflösen wollte. Nicht belegt sind der anschließende Verbindungsaufbau und ein Seitenaufruf durch eine Person. 3. Weil NAT viele Geräte hinter einer Adresse bündelt und DHCP Adressen nur für begrenzte Zeit vergibt. 4. `tls.handshake.type == 1`. 5. Mögliche Ursachen sind ein Cache im Resolver, eine fest eingetragene Adresse in der Hosts-Datei, eine Anfrage über einen anderen Weg wie DNS über HTTPS, oder ein Mitschnitt auf der falschen Schnittstelle. Prüfen lässt sich das über

Resolver-Konfiguration, die Hosts-Datei und einen zweiten Mitschnitt auf allen Schnittstellen.

Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen kontrollierten Mitschnitt erstellt und mit einem Hashwert gesichert hast,
- [ ] HTTP-Anfragen, DNS-Anfragen und TLS-Hostnamen mit tshark gefiltert hast,
- [ ] erklären kannst, was IP-Adresse, DNS-Anfrage und SNI jeweils belegen und was nicht,
- [ ] eine Verbindung mit ihrer Aussagegrenze dokumentieren kannst.

Kapitel 10: Praxisprojekt – Eine simulierte Untersuchung

Lernziele

- Du führst eine kleine, vollständige Untersuchung von der Beweissicherung bis zum Bericht durch.
- Du verbindest die Methoden aus den Kapiteln 3 bis 9 zu einem nachvollziehbaren Ablauf.
- Du trennst im Abschlussbericht Beobachtungen, Schlussfolgerungen und nicht Nachweisbares.

Der Fall

Eine fiktive Organisation stellt fest, dass auf einem Testrechner ein USB-Stick angeschlossen war, der dort nicht hingehört. Der Auftrag lautet, drei Fragen zu beantworten: Wurde der Stick an diesem Rechner angeschlossen, und wann? Wurden Dateien vom Stick kopiert oder auf ihn übertragen? Gibt es Hinweise auf eine Netzwerkverbindung, die im Zusammenhang steht?

Das Projekt ist bewusst klein gehalten. Alle Daten entstehen in deiner eigenen Laborumgebung, und jede Aufgabe baut auf der vorigen auf. Die Fallnummer lautet `BM-2026-0100`, die Beweismittel erhalten Nummern im Schema aus Kapitel 3.

WARNUNG – Sicherheitsrisiko

Das Projekt verwendet ausschließlich eigene Testsysteme und einen Testdatenträger, den du selbst eingerichtet hast. Verwende keine realen Daten und keine Geräte, die dir nicht gehören. Vertausche in keinem Befehl Quelle und Ziel (siehe Kapitel 4).

Umgebung

Du brauchst eine Linux-VM als Untersuchungssystem, einen Testrechner, auf dem der Stick angeschlossen wird, und einen eigenen USB-Stick. Für Aufgabe 5 kann der Testrechner dieselbe Linux-VM sein. Lege vor Beginn für jede VM einen Snapshot an. Die Dokumentation der Befehle erfolgt in einer Protokolldatei `protokoll.md`, die du von Anfang an führst.

Diagramm 10.1: Ablauf des Projekts

```
Aufgabe 1  Auftrag formulieren
Aufgabe 2  Beweismittel erfassen (Kapitel 3)
Aufgabe 3  Image erstellen und prüfen (Kapitel 4, 5)
Aufgabe 4  Partitionsstruktur lesen (Kapitel 4)
Aufgabe 5  Systemspuren auswerten (Kapitel 6, 7)
Aufgabe 6  Timeline erstellen (Kapitel 8)
Aufgabe 7  Netzwerkmitschnitt auswerten (Kapitel 9)
Abschlussbericht
```

Aufgabe 1: Untersuchungsauftrag formulieren

Lernziel: einen Auftrag mit klar begrenztem Scope schreiben (Kapitel 2). **Voraussetzungen:** keine technischen. **Aufgabenstellung:** Formuliere den Auftrag mit den drei Fragen oben. Lege fest, welche Systeme untersucht werden, für welchen Zeitraum, und wer über Erweiterungen entscheidet. **Erwartetes Ergebnis:** Ein Auftrag von höchstens einer halben Seite, in dem jede Frage beantwortbar ist. **Überprüfung:** Kannst du nach dem Projekt zu jeder Frage sagen, ob sie beantwortet wurde? **Fehlerquelle:** Der Scope ist „alles auf dem Rechner“. Begrenze ihn auf den Zeitraum und die Geräte aus dem Auftrag.

Aufgabe 2: Beweismittel erfassen

Lernziel: den Stick mit Zustand, Zeitangabe und Nummer dokumentieren (Kapitel 3). **Voraussetzungen:** dein Testdatenträger, ein Protokoll. **Aufgabenstellung:** Erfasse den Stick mit dem Formular aus Kapitel 3, Übung A. Vergib die Nummer `BM-2026-0100-01`. Halte Seriennummer, Kapazität und den Zustand bei der Übernahme fest, mit Zeit in UTC. **Erwartetes Ergebnis:** Ein vollständig ausgefülltes Formular, dessen Seriennummer mit dem Aufdruck oder der Ausgabe von `lsblk -o NAME,SIZE,MODEL,SERIAL` übereinstimmt. **Fehlerquelle:** Der Stick wird vor dem Erfassen angeschlossen und in das System eingehängt. Erfasse ihn vorher.

Aufgabe 3: Image erstellen und prüfen

Lernziel: ein Bit-für-Bit-Image mit Hashnachweis erstellen (Kapitel 4 und 5). **Voraussetzungen:** Stick aus Aufgabe 2, angeschlossen über einen Write-Blocker oder mit schreibgeschütztem Zugriff, wie in Kapitel 3 beschrieben. **Aufgabenstellung:** Identifiziere den Stick anhand von Größe und Seriennummer. Erstelle das Image, berechne den Hashwert von Stick und Image und vergleiche beides.

Befehle zu Aufgabe 3 (in dieser Reihenfolge)

```
$ lsblk -o NAME,SIZE,MODEL,SERIAL,TRAN,RO
$ sudo blockdev --getro /dev/sdX
$ sudo sha256sum /dev/sdX
$ sudo dd if=/dev/sdX of=BM-2026-0100-01.raw bs=4M
    conv=noerror,sync status=progress
$ sha256sum BM-2026-0100-01.raw
```

Erklärung: Zuerst werden Gerät und Schreibschutzstatus kontrolliert, bevor der Datenträger gelesen wird. Danach berechnest du den Hashwert des Originals, anschließend erstellst du das Image und berechnest dessen Hashwert. `/dev/sdX` ist in beiden Fällen die Quelle des Lesevorgangs, `BM-2026-0100-01.raw` das Ziel. Die Reihenfolge der Argumente bei `dd` bleibt unverändert: `if` vor `of`, Quelle und Ziel sind vor dem Start laut Protokoll zu bestätigen. **Erwartetes Ergebnis:** Die beiden Hashwerte stimmen überein. Meldet `blockdev` den Wert `1`, zeigt der Linux-Kernel den Datenträger als schreibgeschützt. Das ist ein Hinweis, aber kein

eigenständiger Nachweis, dass ein vorgeschalteter Write-Blocker korrekt arbeitet (siehe Kapitel 3). **Typischer Fehler:** `if` und `of` vertauscht. Prüfe jeden Befehl vor dem Start. **Fehleranalyse:** Die Hashwerte unterscheiden sich? Prüfe mit `cmp`, ob Lesefehler aufgetreten sind (`dd` gibt sie aus). Fehlerhafte Blöcke sind im Protokoll zu vermerken.
Sicherheitsbetrachtung: Ab hier arbeitest du nur noch am Image. Der Stick bleibt versiegelt und wird nicht erneut angeschlossen.

Aufgabe 4: Partitionsstruktur lesen

Lernziel: die Partitionstabelle eines Images ohne Einhängen lesen (Kapitel 4, Übung C). **Voraussetzungen:** Image aus Aufgabe 3. **Aufgabenstellung:** Hänge das Image schreibgeschützt ein, lies die Partitionstabelle und löse die Einhängung wieder.

Befehle zu Aufgabe 4

```
$ sudo losetup -r -f --show BM-2026-0100-01.raw
$ sudo fdisk -l /dev/loopN
$ sudo losetup -d /dev/loopN
```

Erwartetes Ergebnis: Die Ausgabe zeigt den Typ der Partitionstabelle (`dos` oder `gpt`) und die Partitionen des Sticks. Hat der Stick keine Tabelle, ist das ein Befund, der ins Protokoll gehört. **Fehlerquelle:** Das Loop-Gerät wird nicht wieder gelöst. Prüfe mit `losetup -a` und löse es nach der Übung.

Aufgabe 5: Systemspuren auswerten

Lernziel: Protokolle und Metadaten auf dem Testrechner lesen, ohne sie zu verändern (Kapitel 6 und 7). **Voraussetzungen:** Testrechner mit dem Stick aus Aufgabe 2, angeschlossen in einem Zeitraum, den du notierst.
Aufgabenstellung: Ermittle auf dem Testrechner, ob der Stick angeschlossen wurde und wann. Verwende dazu die Auswertung der USB-Gerätehistorie aus Kapitel 6 (Registry-Schlüssel USBSTOR und `setupapi.dev.log` auf Windows, auf Linux das Journal mit den Einträgen zum Speichermedium). Lies außerdem die Metadaten der Datei, die auf den Stick kopiert wurde, und vergleiche die Zeitstempel.

Befehle zu Aufgabe 5 (Linux)

```
$ journalctl --utc --since "today" --no-pager | grep -iE
    "usb|sd[a-z]" | head -n 40
$ stat -c '%y | %x | %z | %n' Dateiname
```

Erwartetes Ergebnis: Journaleinträge zum Anschluss des Sticks mit UTC-Zeitstempel, und die Zeitstempel der kopierten Datei. Auf Windows liefert die USBSTOR-Auswertung aus Kapitel 6, Übung C, die Seriennummer des Sticks. **Typischer Fehler:** Die Journalausgabe wird ohne `--utc` gelesen, und die Zeiten werden später mit UTC-Angaben vermischt. **Fehleranalyse:** Enthält das Journal keinen Eintrag zum Anschluss? Nicht jedes System protokolliert den Anschluss von Speichermedien im Journal. Dann fehlt die Quelle, und das ist im Bericht als Grenze festzuhalten, nicht als Beleg dafür, dass nichts angeschlossen war. **Aussagegrenze:** Der Eintrag zeigt, dass der Stick angeschlossen wurde. Er zeigt nicht, was danach geschah.

Aufgabe 6: Timeline erstellen

Lernziel: Zeitangaben aus Dateien und Protokollen in einer gemeinsamen Liste zusammenführen (Kapitel 8). **Voraussetzungen:** Die Protokolldateien aus Aufgabe 5, das Testverzeichnis mit den kopierten Dateien. **Aufgabenstellung:** Erstelle eine Timeline mit der Methode aus Kapitel 8, Übung E. Jede Zeile trägt ihre Quelle.

Befehle zu Aufgabe 6

```
$ journalctl --utc --since "today" -o short-iso --no-pager >
    journal_export.txt
$ for f in testdaten/*; do echo "$(date -u -d "@$(stat -c %Y
    "$f")" +%Y-%m-%dT%H:%M:%SZ) DATEI $f"; done >
    quelle_dateien.txt
$ sed -E 's/^(.{19})[^ ]* /\1Z JOURNAL /' journal_export.txt >
    quelle_journal.txt
$ sort quelle_dateien.txt quelle_journal.txt > timeline.txt
```

Erwartetes Ergebnis: `timeline.txt` mit chronologisch sortierten Zeilen, jede beginnt mit einem Zeitstempel in UTC und nennt danach DATEI oder JOURNAL. **Überprüfung:** Der Anschluss des Sticks und die Erstellung der kopierten Datei stehen in der richtigen Reihenfolge. **Fehlerquelle:** Die

Sommerzeitumstellung wird übersehen, wenn der Untersuchungszeitraum sie enthält. Notiere die Zeitzone im Protokoll.

Aufgabe 7: Netzwerkmitschnitt auswerten

Lernziel: einen kontrollierten Mitschnitt erstellen und die Verbindung auswerten (Kapitel 9). **Voraussetzungen:** Testrechner mit Netzwerkzugang, Übungen A und B aus Kapitel 9 als Vorlage. **Aufgabenstellung:** Erzeuge einen HTTP-Verkehr über die Loopback-Schnittstelle, schneide ihn mit und filtere die Anfragen. Notiere den Zeitpunkt und die Anfrage in deiner Timeline. **Erwartetes Ergebnis:** Eine PCAP-Datei mit Hashwert und eine Liste der HTTP-Anfragen mit Zeitstempel, Quelle und Pfad. **Aussagegrenze:** Der Mitschnitt belegt die Verbindung auf dieser Schnittstelle. Er belegt nicht, wer die Anfrage ausgelöst hat.

Abschlussbericht

Der Bericht fasst die Aufgaben 1 bis 7 zusammen. Er ist der Teil des Projekts, an dem sich die Methode zeigt. Verwende die folgende Gliederung. Sie folgt dem Vorgehen aus Kapitel 2 in verkürzter Form:

Gliederung des Abschlussberichts

```
1. Auftrag (Wortlaut aus Aufgabe 1)
2. Beweismittel (Nummern, Zustand, Zeitangaben aus Aufgabe 2)
3. Vorgehen (Befehle, Werkzeuge, Versionen, Hashwerte)
4. Feststellungen (nur Beobachtungen, mit Quelle und Zeit)
5. Interpretation (Schlussfolgerungen, als solche gekennzeichnet)
6. Nicht nachweisbar (was die Quellen nicht belegen)
7. Grenzen der Untersuchung (Lücken, Zeitabweichungen, fehlende
    Protokolle)
8. Anlagen (Protokoll, Timeline, Hashdateien, Mitschnitt)
```

Im Abschnitt Feststellungen steht beispielsweise „Im Journal des Testrechners ist am Tag X um 14:02 UTC ein Anschluss des Speichermediums mit der Seriennummer Y verzeichnet.“ Im Abschnitt Interpretation steht dazu: „Der Anschluss deutet darauf hin, dass der Stick an diesem Rechner benutzt wurde.“ Die Trennung zwischen beiden Abschnitten ist die Qualität des Berichts.

Qualitätsprüfung des Projekts

Prüfe deinen Bericht anhand der folgenden Punkte. Jeder Punkt, der nicht zutrifft, ist eine Nacharbeit.

- Jede Zeitangabe nennt Zeitzone oder steht in UTC.
- Jede Feststellung nennt ihre Quelle.
- Jeder Befund ist entweder als Beobachtung oder als Schlussfolgerung gekennzeichnet.
- Jeder Hashwert ist dokumentiert, und die Prüfung der Hashwerte ist beschrieben.
- Die Grenzen der Untersuchung stehen im Bericht, auch dort, wo sie unbequem sind.
- Keine Aussage geht über die Quellen hinaus.

Kontrollfragen

1. Warum wird der Stick vor dem Erfassen nicht eingehängt? (leicht)
2. Welche zwei Hashwerte vergleichst du in Aufgabe 3, und was sagt ihr Vergleich aus? (leicht)
3. Der Journaleintrag zeigt einen Anschluss, aber keine Dateiübertragung. Was darf im Bericht stehen? (mittel)
4. Warum muss die Sommerzeitumstellung im Protokoll vermerkt werden, auch wenn sie den Befund nicht zu verändern scheint? (mittel)
5. Dein Bericht formuliert: „Die Person hat den Stick bewusst benutzt.“ Was fehlt dieser Aussage, und wie wäre sie korrekt zu formulieren? (schwer)

Lösungshinweise: 1. Ein Einhängen kann Zeitstempel und Verzeichnisse verändern, bevor die Erfassung abgeschlossen ist. 2. Den Hash des Sticks und den des Images. Ihre Übereinstimmung zeigt, dass das Image den Datenträger zum Zeitpunkt der Sicherung wiedergibt, nicht, dass der Stick vorher unverändert war. 3. Dass der Stick angeschlossen war, nicht dass Daten übertragen wurden. 4. Eine Zeitangabe ohne Zeitzone kann in der Umstellungsnacht zweideutig sein und die Reihenfolge verfälschen, auch wenn das im konkreten Fall unbedeutend ist. 5. Die Aussage vermischt

Beobachtung und Absicht. Korrekt wäre: „Im Journal ist ein Anschluss des Speichermediums verzeichnet. Ob er bewusst erfolgte, lässt sich aus diesen Daten nicht ableiten.“

Lernzielkontrolle

Du hast das Projekt bestanden, wenn du:

- [ ] von der Beweissicherung bis zum Hashnachweis einen Testdatenträger vollständig dokumentiert hast,
- [ ] eine Timeline mit Quellenangabe und UTC-Zeitstempeln erstellt hast,
- [ ] einen Bericht geschrieben hast, in dem Beobachtung, Schlussfolgerung und Nichtnachweisbares getrennt sind,
- [ ] die Grenzen deiner eigenen Untersuchung benennen kannst.

> **Zusammenfassung**
>
> • Ein vollständiges Projekt verbindet Beweissicherung, Hashnachweis, Systemauswertung, Timeline und Netzwerkspuren.
> • Jede Aufgabe baut auf dem Ergebnis der vorigen auf, und jede Zeitangabe bekommt eine Zeitzone.
> • Der Bericht trennt, was beobachtet wurde, was daraus folgt und was sich nicht belegen lässt.

Anhang

Forensischer Spickzettel

Die Reihenfolge gilt für jede Untersuchung, unabhängig vom Werkzeug.

1. Auftrag und Scope schriftlich festhalten (Kapitel 2).
2. Beweismittel identifizieren, Zustand und Zeit dokumentieren, mit Nummer versehen (Kapitel 3).
3. Schreibschutz prüfen, bevor etwas angeschlossen wird (Kapitel 3 und 4).
4. Flüchtige Daten vor allen anderen sichern (Kapitel 1).
5. Hashwert des Originals bilden, dann das Image erstellen, dann den Hashwert des Images bilden (Kapitel 4 und 5).
6. Ab hier nur am Image arbeiten. Das Original bleibt versiegelt (Kapitel 4).
7. Spuren auswerten, Zeiten in UTC umrechnen, Timeline mit Quellen erstellen (Kapitel 6 bis 8).
8. Netzwerkspuren nur aus berechtigten Mitschnitten auswerten (Kapitel 9).
9. Beobachtungen, Schlussfolgerungen und Nichtnachweisbares getrennt dokumentieren (Kapitel 2 und 10).

Linux-Befehle dieses Buches

Befehle: Datenträger und Image

```
$ lsblk -o NAME,SIZE,MODEL,SERIAL,TRAN,RO   # Datenträger mit
    Seriennummer und Schreibschutz
$ sudo blockdev --getro /dev/sdX            # 1 =
    schreibgeschützt, 0 = beschreibbar
$ sudo dd if=/dev/sdX of=beweis.raw bs=4M conv=noerror,sync
    status=progress
$ sudo losetup -r -f --show beweis.raw      # Image
    schreibgeschützt als Schleifengerät
$ sudo fdisk -l /dev/loopN                  # Partitionstabelle
    lesen
```

```
$ sudo losetup -d /dev/loopN                # Schleifengerät lösen
```

Befehle: Hashwerte und Vergleich

```
$ sha256sum datei                           # SHA-256 berechnen
$ sha256sum datei > datei.sha256            # Hashnachweis
    speichern
$ sha256sum -c datei.sha256                 # gegen Nachweis
    prüfen (OK oder FAILED)
$ cmp datei1 datei2                         #
    Byte-für-Byte-Vergleich, nennt die erste Abweichung
$ find ordner -type f -exec sha256sum {} + > hashliste.sha256
```

Befehle: Zeit, Metadaten und Dateien

```
$ date -u                                   # aktuelle Zeit in UTC
$ timedatectl status                        # Zeitzone und
    Synchronisationsstatus
$ stat -c '%y | %x | %z | %n' datei        # Änderung des Inhalts,
    Zugriff, Metadaten, Name
$ find /etc -type f -mtime -7 -ls           # Dateien mit
    geändertem Inhalt in den letzten 7 Tagen
```

Befehle: Protokolle

```
$ journalctl --utc --since "1 hour ago" --no-pager   # Journal in
    UTC
$ journalctl --utc _COMM=sshd --no-pager             # Einträge
    des SSH-Prozesses
$ journalctl --directory=/pfad/zum/journal           # Journal aus
    einer Kopie lesen
$ sudo zgrep " install " /var/log/dpkg.log*          #
    Paketinstallationen, auch komprimiert
$ getent passwd                                      # Konten
    inkl. externer Quellen
```

Befehle: Netzwerk

```
$ sudo tcpdump -i lo -w datei.pcap port 8080        # Mitschnitt
    auf Loopback
$ tshark -r datei.pcap -Y http.request              #
    HTTP-Anfragen anzeigen
```

```
$ tshark -r datei.pcap -q -z follow,tcp,ascii,0     # TCP-Stream
    als Text
```

Windows-PowerShell-Befehle dieses Buches

Befehle: Hash und Dateien

```
PS> Get-FileHash -Path .\beweis.raw -Algorithm SHA256
PS> Get-Item .\datei.txt | Select-Object Name, LastWriteTimeUtc,
    CreationTimeUtc
PS> Get-ChildItem C:\Windows\Prefetch -Filter *.pf
```

Befehle: Ereignisse und Registry

```
PS> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624}
    -MaxEvents 10
PS> Get-WinEvent -Path .\Security.evtx -MaxEvents 20
PS> reg query "HKLM\SYSTEM\CurrentControlSet\Enum\USBSTOR" /s
PS> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624}
    -MaxEvents 1 |
    ForEach-Object { $_.TimeCreated.ToUniversalTime() }
```

Hash-Befehle im Überblick

Verfahren	Ausgabelänge	Einsatz in diesem Buch
SHA-256 (`sha256sum`, `Get-FileHash -Algorithm SHA256`)	64 Hexadezimalzeichen	Standard für Nachweise
SHA-512 (`sha512sum`)	128 Hexadezimalzeichen	ergänzend
MD5 (`md5sum`)	32 Hexadezimalzeichen	nur zum Vergleich mit älteren Dokumenten
SHA-1	40 Hexadezimalzeichen	nur zum Vergleich mit älteren Dokumenten

Wireshark- und tshark-Filter

Filter	Zeigt
`http.request`	HTTP-Anfragen
`dns.flags.response == 0`	DNS-Anfragen, keine Antworten
`dns.qry.name`	gefragter DNS-Name
`tls.handshake.type == 1`	ClientHello der TLS-Verbindungen
`tls.handshake.extensions_server_name`	SNI-Hostname im Handshake
`tcp.stream`	Nummer des TCP-Streams
`ip.addr == 127.0.0.1`	Verkehr mit einer bestimmten Adresse

Formular: Chain of Custody

Feld	Eintrag
Beweismittelnummer	
Gerätetyp, Hersteller, Modell	
Seriennummer	
Kapazität	
Zustand bei Übernahme (Gehäuse, Siegel, Anschlüsse)	
Datum und Uhrzeit mit Zeitzone	
Herkunft des Gerätes	
Übernommen von (Name, Funktion, Unterschrift)	
Verwahrort und Verschlussnummer	

Formular: Übergabeprotokoll

Datum und Uhrzeit (UTC)	Übergeben von	Übernommen von	Zweck	Zustand des Verschlusses	Unterschriften

Beweismittelverzeichnis

Nummer	Gerät	Seriennummer	Erfasst am (UTC)	Verwahrort	Status (Original, Image, Analysekopie)

Untersuchungsprotokoll

Führe das Protokoll von der ersten Minute an. Jeder Eintrag enthält Datum und Uhrzeit mit Zeitzone, Name der Person, Befehl oder Handlung, Werkzeug mit Version, Ergebnis und, falls zutreffend, Fehler oder Abweichungen.

Zeit (UTC)	Person	Handlung oder Befehl	Werkzeug und Version	Ergebnis	Bemerkung

Muster eines Abschlussberichts

Der Bericht folgt der Gliederung aus Kapitel 10. Ein Mustertext für jeden Abschnitt:

1. **Auftrag:** Wortlaut der Untersuchungsfrage, Zeitraum, betroffene Geräte, Entscheidungsweg für Erweiterungen.

2. **Beweismittel:** Nummern, Beschreibung, Zustand bei Übernahme, Chain of Custody mit Verweis auf das Protokoll.

3. **Vorgehen:** Schritte in der Reihenfolge, in der sie ausgeführt wurden, mit Werkzeugen und Versionen.

4. **Feststellungen:** nur Beobachtungen, jede mit Quelle und Zeit in UTC.

5. **Interpretation:** Schlussfolgerungen, jede als solche gekennzeichnet und mit den Feststellungen verknüpft, auf die sie sich stützt.

6. **Nicht nachweisbar:** Fragen, die die Quellen nicht beantworten.

7. **Grenzen:** Lücken in den Protokollen, Zeitabweichungen, fehlende Quellen, Fehler im Vorgehen.

8. **Anlagen:** Protokoll, Timeline, Hashnachweise, Mitschnitte.

Glossar

Acquisition: Erstellung einer vollständigen Kopie eines Datenträgers, des forensischen Images (Kapitel 2, 4).

Anzeigefilter: Ausdruck, der in Wireshark oder tshark bestimmt, welche Pakete angezeigt werden (Kapitel 9).

Beobachtung: Feststellung, die sich unmittelbar aus einer Quelle ergibt und mit Zeit und Quelle belegt ist (Kapitel 2).

Chain of Custody: lückenlose Dokumentation, wer ein Beweismittel wann in Besitz hatte und zu welchem Zweck es übergeben wurde (Kapitel 3).

DNS-Anfrage: Abfrage eines Rechners nach der Adresse zu einem Namen, im Mitschnitt sichtbar (Kapitel 9).

dd: Kommandozeilenwerkzeug, das Blöcke liest und schreibt, geeignet für Bit-für-Bit-Kopien (Kapitel 4).

E01 (Expert Witness Format): Imageformat mit Metadaten und Prüfsummen in der Datei (Kapitel 4).

Ereignis-ID: Nummer, mit der Windows einen Vorgang im Ereignisprotokoll kennzeichnet (Kapitel 6).

Flüchtige Daten: Daten, die beim Abschalten verloren gehen, etwa der Arbeitsspeicher (Kapitel 1).

GPT (GUID Partition Table): Partitionstabelle mit Kopf und Sicherungskopie am Ende des Datenträgers (Kapitel 4).

Hashwert: fester, deterministischer Fingerabdruck von Daten, der Übereinstimmung belegt, nicht Unversehrtheit (Kapitel 5).

Image (forensisches): bitgenaue Kopie eines Datenträgers, einschließlich unbelegter Bereiche (Kapitel 4).

Incident Response: organisierter Ablauf der Reaktion auf einen Sicherheitsvorfall (Kapitel 1).

Inode: Verwaltungsstruktur eines Dateisystems unter Linux, die Metadaten einer Datei speichert (Kapitel 7).

journald: Protokolldienst von systemd, der das Journal schreibt (Kapitel 7).

Schleifengerät (Loop-Gerät): Einhängung einer Datei als Blockgerät, hier schreibgeschützt (Kapitel 4, 10).

MACB: Kurzform für Modified, Accessed, Changed, Birth, die vier Zeitstempel eines Dateisystems (Kapitel 8).

MBR (Master Boot Record): Partitionstabelle im ersten Sektor des Datenträgers (Kapitel 4).

NTP (Network Time Protocol): Protokoll zur Synchronisation von Systemuhren (Kapitel 8).

Nichtflüchtige Daten: Daten, die auf einem Datenträger erhalten bleiben (Kapitel 1).

PCAP: Dateiformat für Netzwerkmitschnitte, mit Wireshark und tshark lesbar (Kapitel 9).

Prefetch: Windows-Mechanismus für den Programmstart, mit Dateien im Verzeichnis `C:\Windows\Prefetch` (Kapitel 6).

Preservation: Maßnahmen, mit denen das Original und die Kopie in ihrem Zustand erhalten bleiben (Kapitel 2).

Reproduzierbarkeit: Eigenschaft einer Untersuchung, deren Schritte so dokumentiert sind, dass eine andere Person zum gleichen Befund kommen kann (Kapitel 2).

Registry-Hive: Datei, die einen Teil der Windows-Registry speichert, etwa `SYSTEM` oder `NTUSER.DAT` (Kapitel 6).

Schlussfolgerung: Deutung von Beobachtungen, die weitere Belege braucht und als solche gekennzeichnet werden muss (Kapitel 2).

Scope: festgelegter Umfang einer Untersuchung nach Systemen, Zeitraum und Fragestellung (Kapitel 2).

SNI (Server Name Indication): Feld im TLS-Handshake, das den Zielnamen überträgt und deshalb im Mitschnitt sichtbar ist (Kapitel 9).

Slack Space: ungenutzter Rest eines Clusters oder Blocks nach dem Ende einer Datei, kann Reste früherer Inhalte enthalten (Kapitel 4).

Timeline: zeitlich sortierte Liste von Ereignissen aus verschiedenen Quellen, jede Zeile mit Quellenangabe (Kapitel 8).

Unbelegter Speicher: Bereiche, die das Dateisystem als frei markiert, aber noch Reste enthalten können (Kapitel 4).

USBSTOR: Registry-Schlüssel mit Angaben zu USB-Massenspeichern (Kapitel 6).

Write-Blocker: Hardware oder Schnittstelle, die Schreibzugriffe auf einen Datenträger unterdrückt (Kapitel 3).

Quellenverzeichnis

Die folgenden Dokumente bilden die Grundlage der Aussagen dieses Buches. Prüfe vor dem Einsatz die jeweils aktuelle Fassung.

Normen und Empfehlungen

• ISO/IEC 27037: Leitlinien für die Identifizierung, Sammlung, Akquisition und Sicherung digitaler Beweismittel. Internationale Norm, erstmals 2012 veröffentlicht.

• NIST Special Publication 800-86: Guide to Integrating Forensic Techniques into Incident Response. National Institute of Standards and Technology, USA. Dokumentation unter nist.gov.

• RFC 3227: Guidelines for Evidence Collection and Archiving. Internet Engineering Task Force, 2002.

• FIPS 180-4: Secure Hash Standard. National Institute of Standards and Technology.

• FIPS 202: SHA-3 Standard. National Institute of Standards and Technology.

Werkzeugdokumentation

• Dokumentation der GNU-Coreutils (`dd`, `stat`, `sha256sum`, `cmp`, `date`), verfügbar über die Manpages des Systems (`man dd`, `man stat`).

• Dokumentation von util-linux (`lsblk`, `blockdev`, `losetup`, `fdisk`), über `man` und die Hilfeoption `--help`.

• Dokumentation von systemd (`journalctl`, `timedatectl`, `systemctl`), über `man journalctl` und die offizielle Projektdokumentation.

• Wireshark-Dokumentation, insbesondere zu Anzeigefiltern und zu tshark, unter wireshark.org.

• Microsoft Learn, Dokumentation zu `Get-FileHash`, `Get-WinEvent`, `Get-Item` und `reg query`, unter learn.microsoft.com.

• tcpdump-Dokumentation, über `man tcpdump`.

• Plaso-Dokumentation des Projekts, für die optionale Übung in Kapitel 8. Die Optionen unterscheiden sich zwischen Versionen, die Hilfeoption `--help` des jeweiligen Befehls ist maßgeblich.

Fachliteratur und Weiterführendes

Dieses Buch nennt keine Titel aus der Fachliteratur, deren Angaben ich nicht vollständig prüfen konnte. Für eine vertiefte Auseinandersetzung mit Methodik und Rechtsrahmen empfehle ich die genannten Normen und Empfehlungen sowie die Dokumentation der eingesetzten Werkzeuge.

Verzeichnis der Befehlsbeispiele

Kapitel	Befehle und Beispiele
1	Diagramm 1.1: Sicherungsreihenfolge

Kapitel	Befehle und Beispiele
2	Diagramm 2.1: Phasen einer Untersuchung, Übungen A und B ohne Befehle
3	Diagramm 3.1: Chain of Custody; Übung C: `lsblk`, `blockdev --getro`
4	Diagramme 4.1 und 4.2; Übungen A bis D: `dd`, `cmp`, `sha256sum`, `fdisk -l`, `losetup`
5	Diagramm 5.1; Übungen A bis D: `sha256sum`, `md5sum`, `Get-FileHash`, `dd`, `find`
6	Diagramm 6.1; Übungen A bis D: `Get-WinEvent`, `reg query`, `Get-ChildItem`
7	Diagramm 7.1; Übungen A bis F: `journalctl`, `stat`, `find`, `zgrep`, `getent`
8	Diagramm 8.1; Übungen A bis F: `date`, `timedatectl`, `stat`, `journalctl`, `sed`, `sort`, Plaso
9	Diagramm 9.1; Übungen A bis E: `tcpdump`, `tshark`, `dig`, `curl`
10	Diagramm 10.1; Aufgaben 1 bis 7 mit Befehlen für Image, Schleifengerät, Journal, Timeline und Netzwerk

Index

Die Verweise führen auf Kapitel. Seitengenaue Verweise folgen in der Endfassung.

Acquisition 2 · Anzeigefilter 9 · Auftrag 2, 10 · Befehlshistorie 7 · Beobachtung 2, 6, 10 · Chain of Custody 3, 5 · DNS 9 · dd 4, 5, 10 · E01 4 · Ereignis-ID 6 · Flüchtige Daten 1 · GPT 4 · Hashliste 5 · Hashwert 5, 10 · Image 4, 5, 10 · Incident Response 1 · Inode 7 · journald 7 · Schleifengerät 4 · MACB 8 · MBR 4 · NTP 8 · Nichtflüchtige Daten 1 · PCAP 9 · Prefetch 6 · Preservation 2 · Reproduzierbarkeit 2 · Registry 6 · Schlussfolgerung 2, 6, 10 · Scope 2, 10 · SNI 9 · Slack Space 4 · Schreibschutz 3, 4 · Timeline 8, 10 · Unbelegter Speicher 4 · USBSTOR 6 · UTC 6, 8 · Write-Blocker 3, 4, 10 · Zeitzone 6, 8 · Sommerzeit 8

DIGITALE FORENSIK

Vom ersten Verdacht bis zum nachvollziehbaren Befund

Dieses Praxishandbuch führt in die digitale Forensik ein: vom Untersuchungsprozess über die Beweissicherung und forensische Images bis zu Hashwerten, Windows- und Linux-Spuren, Timelines und Netzwerkspuren. Der Schwerpunkt liegt auf einer Methodik, die Ergebnisse überprüfbar macht.

DAS ERWARTET DICH

- Untersuchungsprozess, Beweissicherung und Chain of Custody
- Forensische Images und Integritätsprüfung mit Hashwerten
- Windows- und Linux-Spuren methodisch auswerten
- Timelines aus Zeitstempeln und Ereignissen erstellen
- Netzwerkspuren aus PCAP-Dateien, mit Praxisprojekt

Für wen?

Einsteiger in die IT-Sicherheit, angehende Fachinformatiker, Systemadministratoren und Blue-Team-Analysten, die Vorfälle nachvollziehbar untersuchen wollen.

Praxishandbuch · Autor: Eugen Becker · Ausgabe November 2026

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.